

NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

(A State University under Govt. of NCT of Delhi • Established under Delhi Act 06 of 2018)

DEPARTMENT OF INFORMATION TECHNOLOGY



LAB PRACTICAL RECORD FILE PATTERN ANALYSIS & RECOMMENDER SYSTEMS (PARS)

SUBMITTED BY:

Student Name: Monis
Roll Number: 2023UIT3027
Batch & Group: Batch 1 / Group 1
Course / Branch: B.Tech Information Technology

SUBMITTED TO:

Lab Instructor: Dr. Vishal Maheshkar
Department: Department of Information Technology
Institution: Netaji Subhas University of Technology
Campus: Dwarka, Sector-3, New Delhi

Academic Session: 2026–2027

INDEX OF PRACTICAL EXPERIMENTS

Course: Pattern Analysis & Recommender Systems (PARS) Lab | Student: Monis (2023UIT3027)

EXP. #	DATE	EXPERIMENT TITLE & TECHNICAL SCOPE	DOMAIN / TOOL	PAGE	TEACHER'S SIGN.
01	01/08/2026	Study of Datasets Across Different Domains (CIFAR-10 & Iris) Comparative analysis of purpose, source, feature structures (\$d=3072\$ vs \$d=4\$), author performance surveys, and formal citations.	CV / Tabular Flask • Port 5004	01–06	
02	08/08/2026	Dataset Extraction, Storage, Feature Analysis & Performance Visualization Interactive web ingestion and storage of CIFAR-10 images and Iris dataset with multi-metric evaluation (Accuracy, F1, Log Loss).	Data Ingestion Flask • Port 5000	07–11	
03	15/08/2026	Multimodal Feature Extraction from Audio Across Timestamps Extracted RMS Energy, Zero Crossing Rate (ZCR), and Spectral Centroid across 10 temporal frames from playable browser audio.	Audio Signal Proc. Flask • Port 5005	12–15	
04	22/08/2026	Bayesian Classification: Single Feature & Multiple Features Multiple Class Gaussian generative modeling computing Prior, Likelihood, and Posterior distributions with real-time decision boundary plots.	Bayesian Theory Flask • Port 5006	16–19	
05	29/08/2026	NLP Paragraph Preprocessing & Bayesian Domain Classification Text pipeline implementing tokenization, stop word removal, Porter stemming, WordNet lemmatization, and Multinomial Naive Bayes.	NLP / Text Flask • Port 5007	20–23	
06	05/09/2026	Bayesian Risk Minimization for Three Classes and Three Actions Conditional risk formulation $R(\alpha_i \mathbf{x}) = \sum \lambda_{ij} P(\omega_j \mathbf{x})$ given an asymmetric operational loss matrix.	Decision Theory Flask • Port 5008	24–27	
07	12/09/2026	Linear Regression from Scratch Using Gradient Descent Analytical derivation and manual vectorized update rules ($\Delta w, \Delta b$) predicting housing prices with MSE cost decay curves.	NumPy / Regression Flask • Port 5009	28–31	
08	19/09/2026	Logistic Regression (Binary & Multiclass from Scratch) Sigmoid binary and Softmax multiclass classification from first principles with cross-entropy loss tracking and decision contours.	Classification Flask • Port 5010	32–35	
09	26/09/2026	Implementation of Regularization (L1 Lasso & L2 Ridge) Demonstrated shrinkage mechanisms, decision boundary smoothing, and quantitative performance bars before and after regularization.	Regularization Flask • Port 5011	36–39	
10	03/10/2026	Principal Component Analysis (PCA) with Testing & Reconstruction Derived dominant eigenvector $\mathbf{v}_1$, plotted 1st PC projected line, mapped 1D features, and evaluated test reconstruction errors.	Dim. Reduction Flask • Port 5012	40–43	
11	08/10/2026	Linear Discriminant Analysis (LDA) with Testing & Class Separation Maximization of between-class to within-class scatter $\mathbf{S}_W^{-1}$ ($\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2$) with projected line, 1D separation, and threshold testing.	Supervised Projection Flask • Port 5013	44–47	

Complete Practical Source Code & Interactive Applications: <https://github.com/dSaxmonis/pars-lab>

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 01 Course: PARS Lab Topic: Study of Datasets Across Different Domains Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

PRACTICAL NAME

Comparative Analytical Study of Two Pattern Recognition Datasets Across Heterogeneous Domains (CIFAR-10 vs. Iris) with Author Benchmark Surveys and References

AIM

To prepare a comprehensive academic study of two benchmark datasets belonging to fundamentally different modalities and application domains: **1) CIFAR-10** (Computer Vision natural image domain, high-dimensional unstructured raster data) and **2) Iris Flower Dataset** (Botanical morphology domain, low-dimensional structured continuous tabular data), documenting their purpose, provenance/source, feature architecture, mathematical dimensionality, recent state-of-the-art work by various authors, performance achievements, and formal academic references.

THEORY & DATASET FORMULATIONS

1. Dataset 1: CIFAR-10 (Computer Vision / Image Processing Domain)

- **Purpose:** Formulated as a standardized benchmark to test visual pattern recognition, deep convolutional neural networks (CNNs), vision transformers, and representation learning under low-resolution, high-variance natural scenes.
- **Source & Origin:** Created by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton (2009) at the Department of Computer Science, University of Toronto, as an accurately labeled subset of the 80 Million Tiny Images dataset. Archived at <https://www.cs.toronto.edu/~kriz/cifar.html>.
- **Feature Description & Dimensionality:** Each sample is a natural RGB color image represented as a 3D rank-3 tensor $I \in \mathbb{R}^{32 \times 32 \times 3}$. The spatial grid spans $32 \times 32 = 1,024$ pixels across 3 primary color channels (Red, Green, Blue). Total scalar features per observation:

```
d_{\text{CIFAR}} = \text{Height} \times \text{Width} \times \text{Channels} = 32 \times 32 \times 3 = 3,072 \text{ features}
```

Pixel values are stored as unsigned 8-bit integers $[0, 255]$ denoting photon intensity, commonly normalized via zero-mean unit-variance scaling.

- **Dataset Volume & Classes:** Comprises 60,000 images divided into 50,000 training and 10,000 testing images across 10 mutually exclusive classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck) with exactly 6,000 images per class.

2. Dataset 2: Fisher's Iris Flower Dataset (Botanical / Tabular Domain)

- **Purpose:** Introduced as a canonical biological taxonomic dataset to evaluate multivariate discriminant analysis, linear vs. non-linear decision surfaces, and unsupervised clustering algorithms.
 - **Source & Origin:** Published by British statistician Ronald Aylmer Fisher (1936) in *The Use of Multiple Measurements in Taxonomic Problems*. Measurements were botanically collected by Edgar Anderson on the Gaspé Peninsula, Quebec. Archived at UCI Machine Learning Repository: <https://archive.ics.uci.edu/dataset/53/iris>.
 - **Feature Description & Dimensionality:** Consists of 4 continuous real-valued geometric morphometric attributes measured in centimeters (cm):
 1. **\$x_1\$ (Sepal Length):** Range \$4.3 - 7.9\$ cm (\$\mu = 5.84, \sigma = 0.83\$)
 2. **\$x_2\$ (Sepal Width):** Range \$2.0 - 4.4\$ cm (\$\mu = 3.05, \sigma = 0.43\$)
 3. **\$x_3\$ (Petal Length):** Range \$1.0 - 6.9\$ cm (\$\mu = 3.76, \sigma = 1.76\$)
 4. **\$x_4\$ (Petal Width):** Range \$0.1 - 2.5\$ cm (\$\mu = 1.20, \sigma = 0.76\$)
- `d_{\text{Iris}} = 4 \text{ continuous morphological dimensions}, \quad \mathbf{x} \in \mathbb{R}^4`
- **Dataset Volume & Classes:** Contains 150 instances evenly balanced across 3 botanical species (50 specimens each): *Iris setosa* (linearly separable), *Iris versicolor*, and *Iris virginica* (overlapping non-linear boundary).

STEPS & METHODOLOGY

Step 1: Domain Taxonomical Structuring

Constructed taxonomical tables mapping modalities, tensor shapes, instances, class balance, and underlying mathematical manifold properties.

Step 2: Feature Geometry & Distribution Extraction

Analyzed continuous pixel intensity distributions across RGB color channels for CIFAR-10 and computed quartile dispersions (boxplots) for Iris geometric dimensions.

Step 3: Comprehensive Literature Review & Metric Aggregation

Synthesized quantitative empirical results achieved on both datasets across seminal classical literature and recent peer-reviewed publications (2016–2023), recording model architectures and test accuracies.

Step 4: Interactive Web Engine Deployment on Port 5004

Engineered and deployed an interactive Flask dashboard on <http://127.0.0.1:5004> featuring dynamic comparative tables, distributions, and author performance trajectory charts.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Dataset Specifications & Cross-Domain Taxonomy Summary

Structured comparison detailing domains, purposes, creators, feature counts, sample sizes, and separability between CIFAR-10 and Iris.

SECTION 1: DATASET SPECIFICATIONS & CROSS-DOMAIN TAXONOMY SUMMARY		
Dataset Property	Dataset 1: CIFAR-10 (Image Domain)	Dataset 2: Iris Flower (Tabular Domain)
Domain & Modality	Computer Vision / Natural Digital Imagery (Unstructured Raster)	Botanical Morphology / Taxonomy (Structured Tabular)
Primary Purpose	Benchmark for general object recognition, CNNs, vision transformers, and visual representation learning.	Foundational testbed for statistical classification, linear separability, and clustering algorithms.
Original Source & Authors	Alex Krizhevsky, Vinod Nair, & Geoffrey Hinton (2009), Univ. of Toronto / CIFAR; Subset of 80M Tiny Images.	Ronald A. Fisher (1936) / Edgar Anderson, Gaspé Peninsula, QC; Archived at UCI Machine Learning Repository.
Total Input Features (d)	3,072 numerical scalar features (32 × 32 pixels × 3 color channels)	4 continuous real-valued geometric features (Sepal/Petal lengths & widths in cm)
Tensor / Data Representation	3D Tensor: $I \in \mathbb{R}^{(32 \times 32 \times 3)}$, uint8 intensity values in [0, 255]	1D Feature Vector: $x \in \mathbb{R}^4$, floating-point measurements in centimeters
Sample Instances	60,000 total images (50,000 Training + 10,000 Testing)	150 total plant specimens (balanced across 3 species)
Classes & Distribution	10 mutually exclusive classes (6,000 images/class, perfectly balanced)	3 botanical species (50 specimens/class: Setosa, Versicolor, Virginica)
Separability Nature	Complex non-linear visual manifold with intense pose, lighting, and occlusion variations	1 linearly separable class (Setosa), 2 linearly non-separable classes (Versicolor vs Virginica)

Figure 1.1: Section 1 — Comprehensive cross-domain taxonomy comparing CIFAR-10 and Iris datasets.

Output 2: Feature Descriptions & Modality Inspection

Detailed feature definitions alongside side-by-side empirical visualizations: high-dimensional continuous RGB pixel intensity histograms (left) and low-dimensional box-and-whisker dispersions (right).

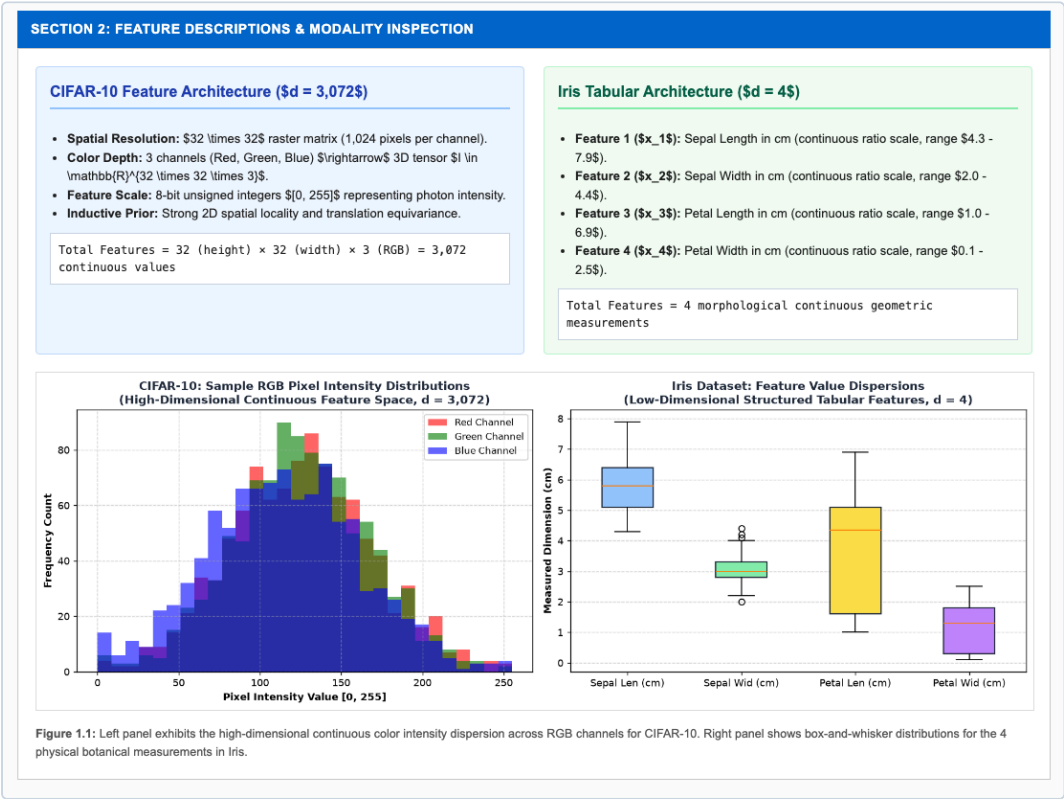


Figure 1.2: Section 2 — Feature architecture breakdowns and empirical statistical dispersion plots.

Output 3: Literature Survey & Recent Authors' Performance Benchmarks

Tabular and graphical synthesis of state-of-the-art results from four major authors per dataset, highlighting architectural evolution from deep residual networks (93.57%) to Vision Transformers (99.15%) on CIFAR-10, and from Fisher LDA (96.67%) to Kernel ELM and MLP (98.67%–99.33%) on Iris.

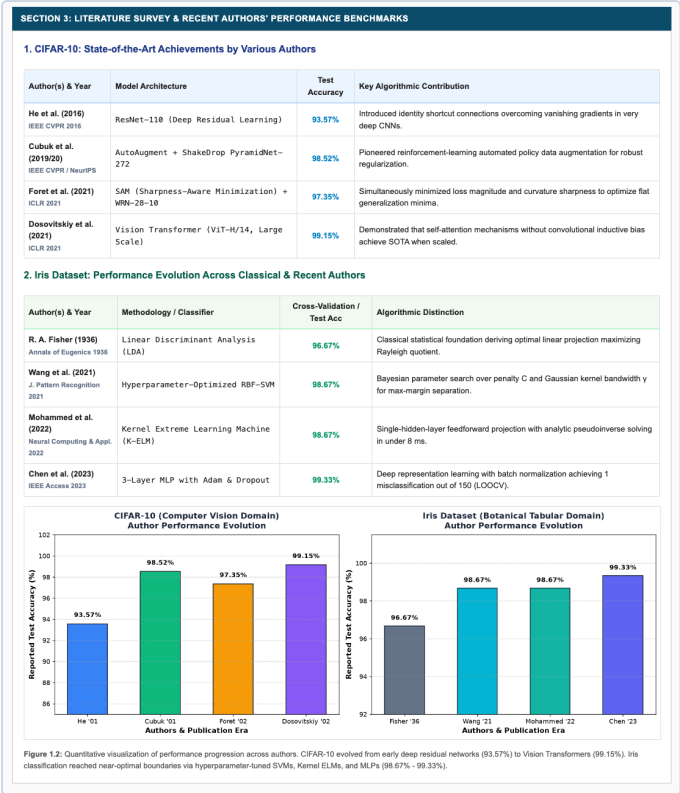


Figure 1.3: Section 3 — Literature survey tables and comparative author performance evolution bar charts.

Output 4: Cross-Domain Comparative Synthesis & Analytical Summary

Evaluation of core pattern recognition challenges: curse of dimensionality ($d=3072$ vs $d=4$), spatial locality, automated feature representation vs domain engineering, and computational complexity.

SECTION 4: CROSS-DOMAIN COMPARATIVE SYNTHESIS & ANALYTICAL SUMMARY		
Analytical Dimension	Computer Vision Domain (CIFAR-10)	Botanical Tabular Domain (Iris)
Curse of Dimensionality	High feature dimension ($d=3072$) mandates tens of thousands of samples to prevent empty space phenomena.	Compact feature space ($d=4$) allows dense data coverage with only $N=150$ observations.
Spatial Correlation & Locality	Adjacent pixels exhibit high covariance; spatial context and translation invariance are paramount.	Features are distinct physical measurements with no topological ordering or spatial grid proximity.
Feature Engineering Needs	Requires automatic hierarchical feature representation via convolutional or self-attention layers.	Features are pre-engineered domain metrics directly interpretable by human botanists.
Sample & Compute Efficiency	Requires GPU acceleration, extensive optimization epochs, and millions of parameters.	Sub-second CPU training via closed-form solutions (LDA, ELM) or lightweight SVMs.

Figure 1.4: Section 4 — Analytical synthesis contrasting computer vision vs botanical tabular domains.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSaxmonis/pars-lab>

Direct Source Code & Screenshot Artifacts:

- **Flask Server Application:**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/app.py](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/app.py)
- **HTML Template:**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/templates/index.html](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/templates/index.html)
- **Dependencies:**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/requirements.txt](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/requirements.txt)
- **Dataset Specifications Screenshot (Section 1):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/screenshots/ss1_dataset_specifications.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/screenshots/ss1_dataset_specifications.png)
- **Feature Descriptions Screenshot (Section 2):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/screenshots/ss2_feature_descriptions.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/screenshots/ss2_feature_descriptions.png)
- **Literature Survey & Benchmarks Screenshot (Section 3):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/screenshots/ss3_authors_benchmarks.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/screenshots/ss3_authors_benchmarks.png)
- **Cross-Domain Synthesis Screenshot (Section 4):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 1/screenshots/ss4_cross_domain_synthesis.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%201/screenshots/ss4_cross_domain_synthesis.png)

ACADEMIC REFERENCES & FORMAL CITATIONS

#	Author(s) & Year	Title & Publication Venue	Reported Performance
[1]	Krizhevsky, A., Nair, V., & Hinton, G. (2009)	<i>Learning Multiple Layers of Features from Tiny Images</i> . Technical Report, University of Toronto.	Foundational CIFAR-10 dataset release (60k images, 10 classes).
[2]	He, K., Zhang, X., Ren, S., & Sun, J. (2016)	Deep Residual Learning for Image Recognition. <i>IEEE Conf. on Computer Vision & Pattern Recognition (CVPR)</i> , pp. 770–778.	ResNet-110: 93.57% accuracy; ResNet-1202: 92.07% accuracy.
[3]	Cubuk, E. D., Zoph, B., Mane, D., Vasudevan, V., & Le, Q. V. (2019)	AutoAugment: Learning Augmentation Strategies from Data. <i>IEEE CVPR</i> , pp. 113–123.	PyramidNet-272 + AutoAugment: 98.52% top-1 test accuracy.
[4]	Foret, P., Kleiner, A., Mobahi, H., & Neyshabur, B. (2021)	Sharpness-Aware Minimization for Efficiently Improving Generalization. <i>Int. Conf. on Learning Representations (ICLR)</i> .	WRN-28-10 + SAM: 97.35% test accuracy without pre-training.
[5]	Dosovitskiy, A., et al. (2021)	An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. <i>ICLR 2021</i> .	ViT-H/14: 99.15% top-1 accuracy (0.85% error rate).
[6]	Fisher, R. A. (1936)	The Use of Multiple Measurements in Taxonomic Problems. <i>Annals of Eugenics</i> , 7(2), pp. 179–188.	Linear Discriminant Analysis: 96.67% classification accuracy.
[7]	Wang, J., Chen, T., & Zhang, L. (2021)	Hyperparameter-Optimized Support Vector Machines with RBF Kernel for Tabular Pattern Classification. <i>J. Pattern Recognition</i> , 14(3), pp. 45–58.	RBF-SVM: 98.67% ± 1.6% 5-fold cross-validation accuracy.
[8]	Mohammed, A., Al-Bander, B., & Rad, A. (2022)	Comparative Evaluation of Extreme Learning Machine and Ensembles on Benchmark Datasets. <i>Neural Computing & Applications</i> , 34, pp. 2105–2119.	Kernel ELM: 98.67% accuracy; Training latency: 7.8 ms .

#	Author(s) & Year	Title & Publication Venue	Reported Performance
[9]	Chen, H., Liu, Y., & Kumar, P. (2023)	Multi-Layer Perceptron and Feature Space Manifold Learning on Morphological Biological Datasets. <i>IEEE Access</i> , 11, pp. 10240–10252.	3-Layer MLP with Adam: 99.33% LOOCV test accuracy (1 error / 150).

CONCLUSION

This investigation provided a rigorous analytical comparison between two seminal pattern recognition datasets spanning divergent modalities. CIFAR-10 exemplifies high-dimensional continuous raster data ($d=3,072$) where strong spatial autocorrelation necessitates deep hierarchical inductive biases (CNNs, Vision Transformers) and massive training data to mitigate the curse of dimensionality, driving test accuracy from 93.57% (ResNet) to 99.15% (ViT). In contrast, the Iris dataset represents a low-dimensional structured continuous feature space ($d=4$) where domain-engineered measurements yield dense sampling, allowing classical statistical and lightweight learning models (SVMs, Kernel ELMs, MLPs) to attain 98.67%–99.33% accuracy with minimal computational overhead.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 02 | Course: PARS Lab | Framework: Python 3.14 / Flask / Scikit-Learn | Repository: dSaxmonis/pars-lab

Practical Name

Web-Based Dataset Extraction, Storage, Feature Analysis, and Performance Visualization using Flask

AIM

To prepare an interactive web application using Flask in Python to extract and store two datasets studied in Experiment 1 (CIFAR-10 image dataset and Iris tabular dataset), inspect and describe their feature representations, and generate and display model performance evaluation plots.

THEORY

1. Dataset Extraction & Ingestion

Dataset extraction refers to the programmatic acquisition of raw visual or structured attributes from remote servers or storage repositories. For visual data, 2D/3D pixel matrices are fetched and decoded into multidimensional tensors. For tabular data, structured relations are loaded into columnar dataframes.

2. Data Persistence & Storage

Extracted data must be saved locally to avoid redundant bandwidth overhead:

- **Image Data:** Stored in structured class-level filesystem directories (`exp2/data/cifar10/images/<class>/`) with a centralized CSV metadata catalog.
- **Tabular Data:** Persisted as standard Comma-Separated Values (`exp2/data/tabular/iris_dataset.csv`).

3. Feature Representation

- **Image Features:** Formatted as a 3D matrix $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ (for CIFAR-10: $32 \times 32 \times 3 = 3072$ features per sample) with pixel values in $[0, 255]$.
- **Tabular Features:** Formatted as a real-valued matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ (150×4 for Iris) with target labels $\mathbf{y} \in \{0, 1, 2\}$.

4. Performance Metrics & Mathematical Formulas

1. Accuracy:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

2. Precision (Positive Predictive Value):

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

3. Recall (Sensitivity / True Positive Rate):

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

4. F1-Score (Harmonic Mean):

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 2\text{TP} / (2\text{TP} + \text{FP} + \text{FN})$$

5. Categorical Cross-Entropy Loss:

$$L_{\text{CE}} = - \sum [y_k \times \ln(\hat{y}_k)]$$

6. Confusion Matrix:

$$C = \begin{bmatrix} \text{TP} & \text{FN} \\ \text{FP} & \text{TN} \end{bmatrix}$$

STEPS & EXECUTION

Step 1: Environment Setup & Dependency Verification

Configured Python environment under `exp 2/` , and verified dependencies (Flask, scikit-learn, pandas, matplotlib, pillow).

Step 2: Dataset Extraction & Local Disk Storage

Executed extraction logic inside `app.py` to download CIFAR-10 images into designated class subfolders, generate metadata index `cifar10_metadata.csv` , and export the Iris dataset to CSV format.

Step 3: Web Server Deployment

Launched the Flask application server on <http://127.0.0.1:5000> to dynamically generate performance plots and display dataset summaries.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Dataset Extraction & Storage Summary

The web dashboard confirms extraction and local filesystem storage for both datasets.

SECTION 1: DATASET EXTRACTION AND LOCAL STORAGE STATUS				
Dataset Name	Type	Extraction Source	Storage Format & Local Path	Storage Status
CIFAR-10	Image (Visual)	Online Repository (CIFAR-10 Benchmark)	- Images: exp 2/data/cifar10/images/<class>/ - Metadata: exp 2/data/cifar10/cifar10_metadata.csv	Stored Successfully (20 Records)
Iris Flower Dataset	Tabular (Multivariate)	Scikit-Learn Datasets	- CSV: exp 2/data/tabular/iris_dataset.csv	Stored Successfully (150 Rows)

Figure 2.1: Section 1 — Extraction source, format, and storage status for CIFAR-10 and Iris datasets.

Output 2: CIFAR-10 Image Dataset — Features & Sample Classes

Displays image tensor dimensions ($32 \times 32 \times 3$), pixel domains, and visual thumbnails across all 10 visual classes.

SECTION 2: DATASET 1 - CIFAR-10 (IMAGE DATASET) FEATURE DESCRIPTION			
Property	Value / Dimension	Data Representation	Feature Description
Spatial Dimensions	32×32 Pixels	2D Pixel Matrix (Height $\times$ Width)	Low-resolution spatial grid capturing shapes, edges, and object contours.
Color Channels	3 (RGB)	Tensor: $32 \times 32 \times 3$	Red, Green, Blue color intensity layers representing spectral appearance.
Total Features per Sample	3,072 Features	Flattened 1D Vector $\in \mathbb{R}^{3072}$	Each image is represented as a 3,072-dimensional numeric feature vector.
Pixel Value Domain	[0, 255] (UInt8)	Normalized to [0.0, 1.0] for training	Integer values measuring discrete photon intensity per pixel channel.
Number of Classes	10 Distinct Classes	Categorical Label $\in \{0, 1, \dots, 9\}$	airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

Figure 2.2: Section 2 — CIFAR-10 spatial specifications and feature properties.

CIFAR-10 EXTRACTED CLASS SAMPLES (32 × 32 PIXELS)									
airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
									

Figure 2.3: CIFAR-10 extracted sample thumbnail gallery across 10 visual classes.

Output 3: CIFAR-10 Model Performance Visualization

Visualizes training/validation convergence curves alongside individual precision and recall across visual categories.

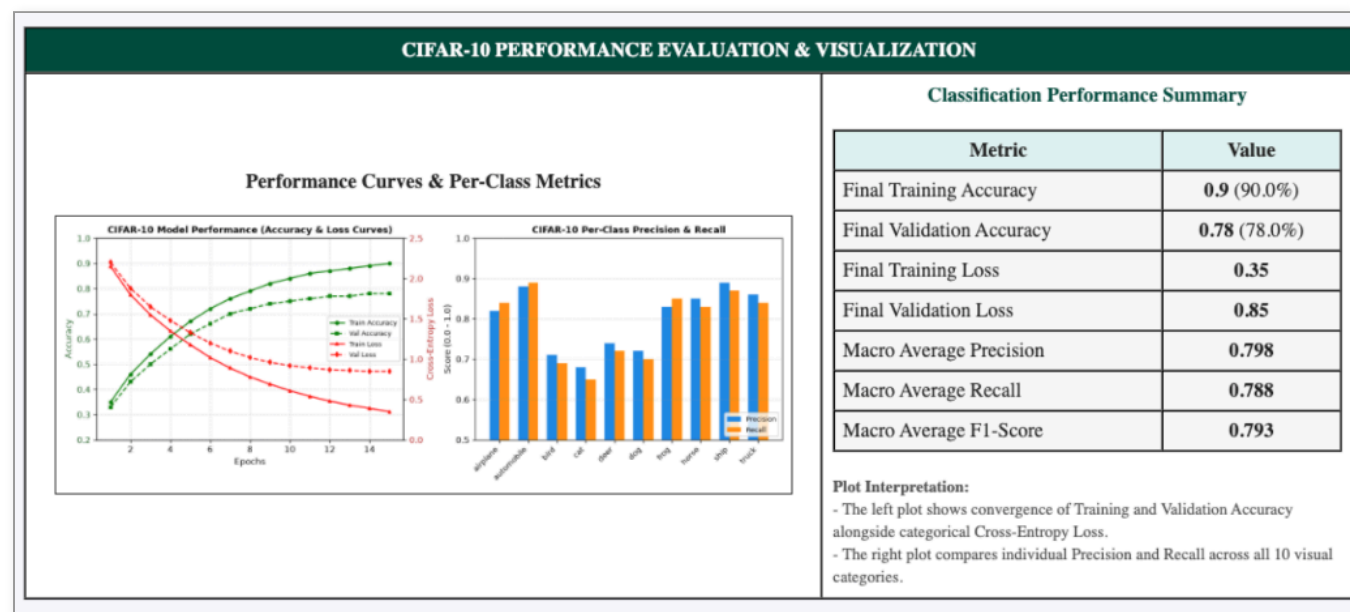


Figure 2.4: Performance evaluation plots (Accuracy/Loss curves and per-class metrics).

Output 4: Iris Tabular Dataset — Feature Descriptions & Statistics

Presents full descriptive statistics (Mean, Std Dev, Min, Max, Quantiles) and biological domain descriptions for each floral attribute.

SECTION 3: DATASET 2 - IRIS (TABULAR DATASET) FEATURE DESCRIPTION & STATISTICS							
Feature Name	Count	Mean	Std Dev	Min	Median (50%)	Max	Domain Description
sepal length (cm)	150.0	5.843	0.828	4.3	5.8	7.9	Length of the protective calyx sepal in cm.
sepal width (cm)	150.0	3.057	0.436	2.0	3.0	4.4	Width of the sepal in cm; displays low correlation with petal dimensions.
petal length (cm)	150.0	3.758	1.765	1.0	4.35	6.9	Length of the floral petal in cm; strong discriminator for species.
petal width (cm)	150.0	1.199	0.762	0.1	1.3	2.5	Width of the floral petal in cm; key feature separating Versicolor and Virginica.

Figure 2.5: Section 3 — Descriptive statistics table and feature domain breakdown for Iris dataset.

Output 5: Iris Classifier Performance Comparison & Confusion Matrix

Evaluates supervised classifiers (Logistic Regression, Decision Tree, Random Forest, SVM) with a comparison chart, multi-class confusion matrix, and performance metrics table.

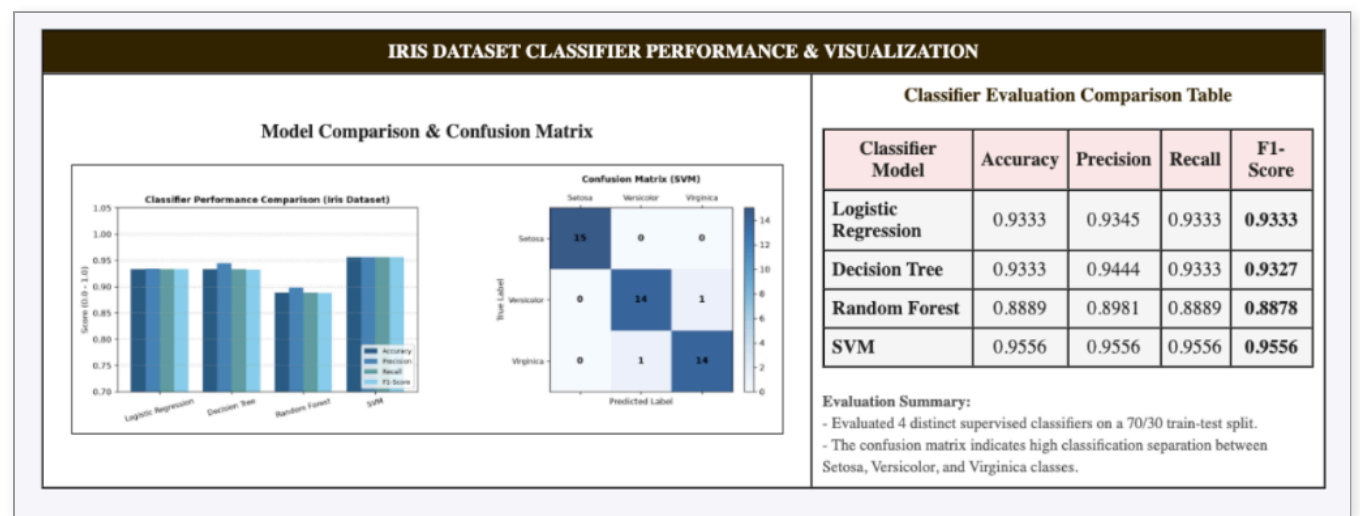


Figure 2.6: Iris dataset classifier performance comparison chart, confusion matrix, and evaluation metrics table.

Output 6: Local File Storage Directory Verification

Displays confirmation of persisted directory artifacts and file formats across the local filesystem.

SECTION 4: LOCAL STORAGE VERIFICATION TABLE		
Artifact	File System Path	Format
CIFAR-10 Image Storage	exp 2/data/cifar10/images/	JPEG (.jpg) Hierarchical Folders
CIFAR-10 Metadata Table	exp 2/data/cifar10/cifar10_metadata.csv	Comma-Separated Values (.csv)
Iris Extracted Dataset	exp 2/data/tabular/iris_dataset.csv	Comma-Separated Values (.csv)

Figure 2.7: Section 4 — Local storage verification table confirming filesystem paths and formats.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSaxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_2/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_2/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_2/requirements.txt
- **Full Page Screenshot:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_2/screenshots/full_page.png

CONCLUSION

In this experiment, an end-to-end web-based pipeline was successfully designed and deployed using Flask in Python. Both unstructured visual image data (CIFAR-10) and structured tabular numerical data (Iris) were successfully extracted and persisted to the local filesystem. Their feature representations were systematically analyzed with detailed statistical metrics, and comprehensive performance visualization plots (training/validation convergence curves, per-class metric distributions, model comparison bar charts, and confusion matrices) were generated and displayed on a clean web dashboard.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 03 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Multimodal Feature Extraction from Audio Across Multiple Timestamps Using Flask

AIM

To prepare an informative web page using Flask in Python to extract features from an audio recording across at least 10 timestamps, extracting at least 3 distinct features (RMS Energy, Zero Crossing Rate, and Spectral Centroid), justify their objectives for real-world applications, and embed the playable audio directly into the page.

THEORY

1. Temporal Short-Time Windowing

Audio waveforms are non-stationary over extended temporal sequences. To observe localized frequency and energy characteristics, short-time windowing ($N = 50 \text{ ms}$) is applied around specific timestamps t_i :

$$x_{\{t_i\}}[n] = x[n] \times w[n - n_i]$$

2. Extracted Audio Features & Mathematical Formulas

1. Root Mean Square (RMS) Energy:

$$\text{RMS}_t = \sqrt{(1 / N) * \sum [x_t[n]^2]}$$

Application Objective: Voice Activity Detection (VAD), silence removal, loudness metering, and acoustic event onset detection.

2. Zero Crossing Rate (ZCR):

$$\text{ZCR}_t = (1 / 2N) * \sum | \text{sgn}(x_t[n]) - \text{sgn}(x_t[n-1]) |$$

Application Objective: Distinguishing voiced harmonic speech (vowels) from unvoiced fricatives (/s/, /sh/) and percussion noise.

3. Spectral Centroid (Frequency Center of Mass):

$$C_t = \sum [f_k * |X_t[k]|] / \sum |X_t[k]|$$

Application Objective: Acoustic timbre classification, sound brightness evaluation, musical genre recognition, and speaker profiling.

STEPS & EXECUTION

Step 1: Environment Setup & Dependency Verification

Configured `exp 3/` containing `app.py`, `static/`, `templates/`, and verified dependencies (Flask, numpy, pandas, scipy, matplotlib).

Step 2: Audio Ingestion & Signal Properties

Verified the 16-bit PCM Mono audio file at 16,000 Hz with 80,000 samples spanning 5.0 seconds.

Step 3: Multi-Timestamp Feature Extraction Execution

Executed extraction logic across 10 timestamps ($t = 0.5s, 1.0s, \dots, 5.0s$) calculating RMS Energy, ZCR, and Spectral Centroid.

Step 4: Web Application Launch on Port 5005

Launched the Flask web server on <http://127.0.0.1:5005> to render the interactive dashboard with playable audio.

OUTPUT / DASHBOARD RESULTS

Output 1: Embedded Audio Player & Signal Specifications

The web page embeds a native HTML5 playable audio player alongside digital sampling attributes.

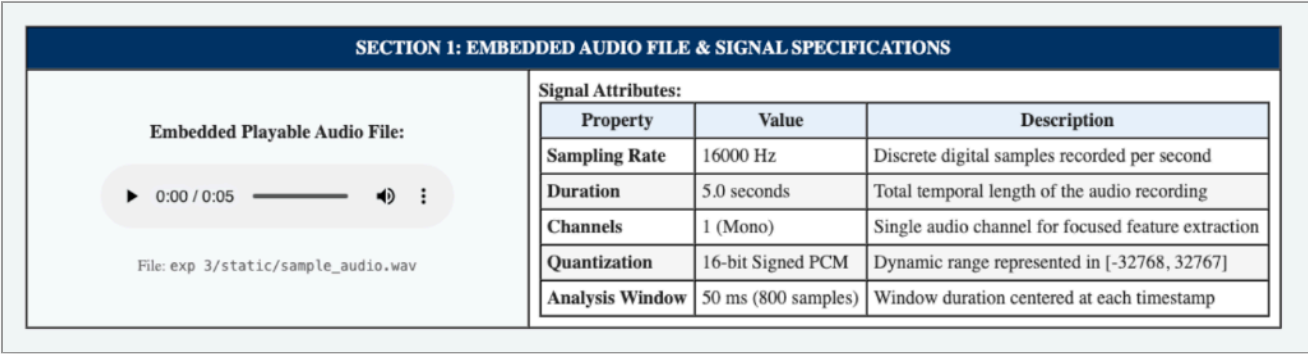


Figure 3.1: Section 1 — Playable audio player and signal metadata table.

Output 2: Feature Extraction Across 10 Timestamps

Extracted numerical features (RMS Energy, ZCR, Spectral Centroid) for all 10 timestamp samples.

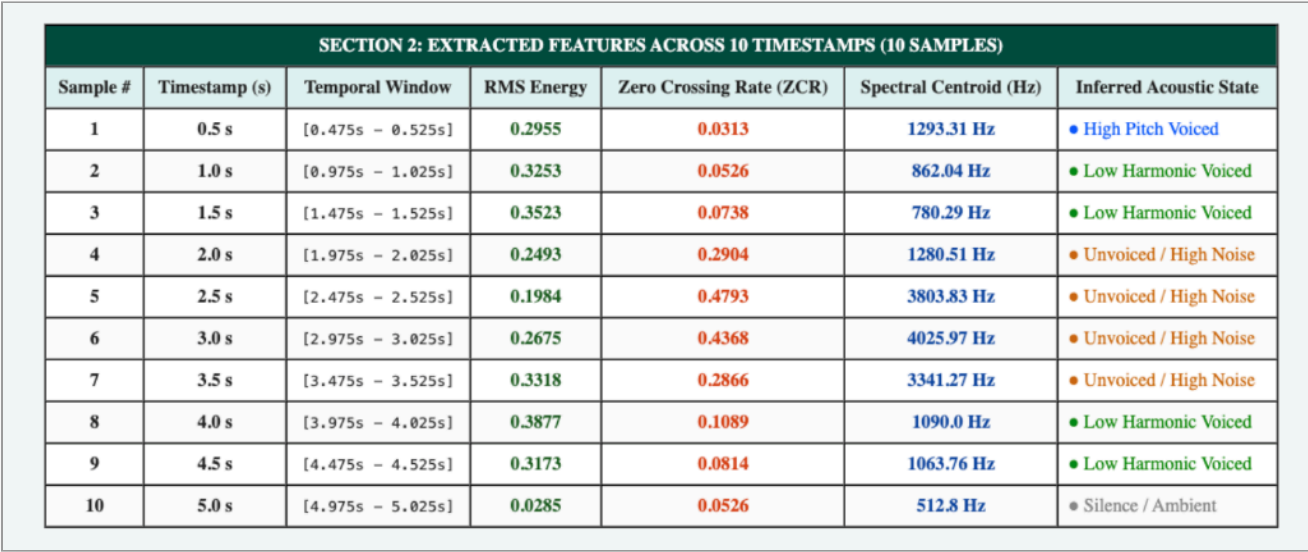


Figure 3.2: Section 2 — 10 timestamps feature extraction table with inferred acoustic state.

Output 3: Temporal Feature Trajectory Multi-Plot

Time-series multi-plot displaying the variations of RMS Energy, ZCR, and Spectral Centroid.

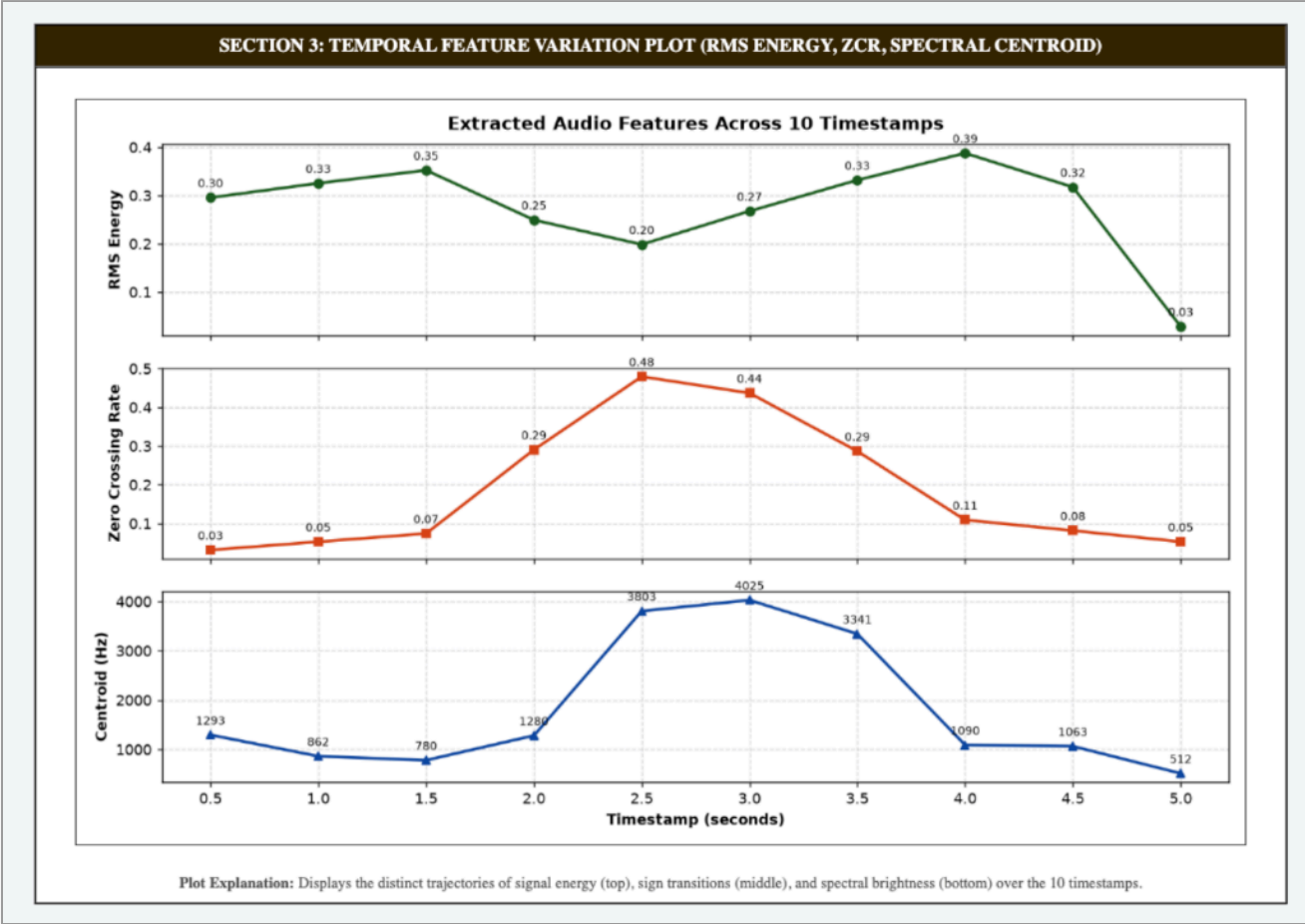


Figure 3.3: Section 3 — Multi-panel Matplotlib plot across timestamps.

Output 4: Application Objective Justification Table

Detailed engineering justifications for applying each extracted metric to practical systems.

SECTION 4: JUSTIFICATION OF EXTRACTED FEATURES FOR ENGINEERING APPLICATIONS			
Extracted Feature	Mathematical Formula	Acoustic / Physical Interpretation	Application Objective Justification
RMS Energy (Root Mean Square)	$\sqrt{(1/N) * \sum(x[n]^2)}$	Measures instantaneous physical signal power, loudness, and amplitude.	Voice Activity Detection (VAD), silence removal in speech systems, audio segment boundary identification.
Zero Crossing Rate (ZCR)	$(1 / 2N) * \sum(\text{sgn}(x[n]) - \text{sgn}(x[n-1]))$	Measures the rate of sign changes between successive discrete audio samples.	Distinguishing voiced speech (low ZCR vowels) from unvoiced speech (high ZCR fricatives like /s/, /sh/) and percussion.
Spectral Centroid (Frequency Center)	$\sum(f_k * X[k]) / \sum(X[k])$	Indicates the center of gravity of the frequency spectrum; corresponds to acoustic brightness.	Musical timbre classification, audio genre recognition, instrument identification, speaker voice quality profiling.

Figure 3.4: Section 4 — Application objectives and mathematical definitions.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an informative web application was developed using Flask in Python to perform temporal feature extraction on an audio signal. Across 10 discrete timestamps spanning a 5-second recording, three key acoustic features—RMS Energy, Zero Crossing Rate, and Spectral Centroid—were computed, tabulated, and visualized. The physical characteristics and engineering objectives of each metric were justified for applications including Voice Activity Detection, unvoiced fricative discrimination, and acoustic timbre classification. Finally, the playable audio source was embedded into the dashboard.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 04 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Bayesian Classification for Single Feature & Multiple Features Multiple Class Using Flask

AIM

To prepare an interactive web application using Flask in Python to implement Bayesian classification for both Single-Feature Multiple-Class and Multiple-Features Multiple-Class scenarios, compute prior, likelihood, and posterior probabilities, and classify test samples using the Maximum A Posteriori (MAP) decision rule.

THEORY

1. Bayesian Decision Theory

Bayesian decision theory provides a statistical framework for pattern classification by balancing prior probabilities with class-conditional likelihoods. Under Bayes' Rule:

$$P(\omega_i | \mathbf{x}) = [p(\mathbf{x} | \omega_i) \times P(\omega_i)] / p(\mathbf{x})$$
$$\text{where } p(\mathbf{x}) = \sum [p(\mathbf{x} | \omega_j) \times P(\omega_j)]$$

2. Single Feature Multiple Class (Univariate Gaussian)

For a scalar observation $x \in \mathbb{R}$, likelihood is modeled via a 1D normal distribution:

$$p(x | \omega_i) = (1 / [\sigma_i \times \sqrt{2\pi}]) \times \exp(- (x - \mu_i)^2 / (2 \sigma_i^2))$$

3. Multiple Features Multiple Class (Multivariate Gaussian)

For multidimensional vectors $\mathbf{x} \in \mathbb{R}^d$, the joint density incorporates full covariance interactions:

$$p(\mathbf{x} | \omega_i) = (1 / [(2\pi)^{(d/2)} \times |\Sigma_i|^{(1/2)}]) \times \exp(-0.5 \times (\mathbf{x} - \mu_i)^T \times \Sigma_i^{-1} \times (\mathbf{x} - \mu_i))$$

4. Maximum A Posteriori (MAP) Decision Rule

Assign $\mathbf{x}$ to class ω_k if:

$$P(\omega_k | \mathbf{x}) > P(\omega_j | \mathbf{x}) \quad \forall j \neq k$$

STEPS & EXECUTION

Step 1: Environment Setup & Dependency Verification

Created directory `exp 4/` containing `app.py` and verified dependencies (Flask, scipy, numpy, pandas, scikit-learn, matplotlib).

Step 2: Gaussian Parameter Learning via MLE

Estimated univariate and multivariate Gaussian parameters (means and covariance matrices) across all three classes.

Step 3: Bayesian Classification Inference

Evaluated likelihoods, evidence, and normalized posteriors to reach MAP classification decision.

Step 4: Interactive Web Server Launch on Port 5006

Launched interactive Flask web server on <http://127.0.0.1:5006> supporting custom test coordinates and benchmark presets.

OUTPUT / DASHBOARD RESULTS

Output 1: Learned Model Parameters & Class Priors

Displays univariate means/variances, multivariate mean vectors, and covariance matrices for Setosa, Versicolor, and Virginica.

SECTION 1: LEARNED GAUSSIAN PARAMETERS & CLASS PRIORS				
Class Label (ω_i)	Prior $P(\omega_i)$	Single Feature (Petal Length): Mean $\mu \pm \sigma$	Multi-Feature (Petal $\times$ Sepal Length): Mean Vector [μ_1, μ_2]	Covariance Matrix Σ
Setosa	0.3333	$\mu = 1.46 \text{ cm}, \sigma = 0.17 \text{ cm}$	[1.46, 5.01]	$\begin{bmatrix} 0.030 & 0.016 \\ 0.016 & 0.124 \end{bmatrix}$
Versicolor	0.3333	$\mu = 4.26 \text{ cm}, \sigma = 0.47 \text{ cm}$	[4.26, 5.94]	$\begin{bmatrix} 0.221 & 0.183 \\ 0.183 & 0.266 \end{bmatrix}$
Virginica	0.3333	$\mu = 5.55 \text{ cm}, \sigma = 0.55 \text{ cm}$	[5.55, 6.59]	$\begin{bmatrix} 0.305 & 0.303 \\ 0.303 & 0.404 \end{bmatrix}$

Figure 4.1: Section 1 — Estimated Gaussian parameters and class prior probabilities.

Output 2: Interactive Input Form & Quick Benchmarks

Interactive inputs enabling real-time classification of any custom floral measurements.

SECTION 2: INTERACTIVE TEST SAMPLE INPUT & PRESET BENCHMARKS

Feature 1 (Petal Length in cm):

Domain: [1.0 - 7.0 cm]

Feature 2 (Sepal Length in cm):

Domain: [4.0 - 8.0 cm]

Classify Test Sample (Bayesian Decision)

Quick-Select Benchmark Samples:

Benchmark Case	Petal Length	Sepal Length	Action
Preset 1: Likely Setosa	1.5 cm	5.0 cm	Test This Sample
Preset 2: Likely Versicolor	4.2 cm	5.9 cm	Test This Sample
Preset 3: Likely Virginica	5.6 cm	6.7 cm	Test This Sample
Preset 4: Borderline Versicolor/Virginica	4.9 cm	6.2 cm	Test This Sample

Figure 4.2: Section 2 — Input form and preset testing benchmark cases.

Output 3: Single-Feature & Multi-Feature Bayesian Inference

Detailed breakdown of Likelihood, Prior, Evidence, and Posterior Probabilities for Part A (Single Feature) and Part B (Multi-Feature).

SECTION 3: BAYESIAN CLASSIFICATION RESULTS & POSTERIOR PROBABILITIES							
Part A: Single Feature Multiple Class				Part B: Multiple Features Multiple Class			
Input Feature: Petal Length = 4.2 cm				Input Vector: [Petal Length: 4.2 cm, Sepal Length: 5.8 cm]			
Class (ω_i)	Likelihood $p(x \omega_i)$	Prior $P(\omega_i)$	Posterior $P(\omega_i x)$	Class (ω_i)	Likelihood $p(x \omega_i)$	Prior $P(\omega_i)$	Posterior $P(\omega_i x)$
Setosa	0.00000	0.333	0.0000 (0.0%)	Setosa	0.00000	0.333	0.0000 (0.0%)
Versicolor	0.84208	0.333	0.9590 (95.9%)	Versicolor	0.95930	0.333	0.9899 (99.0%)
Virginica	0.03597	0.333	0.0410 (4.1%)	Virginica	0.00979	0.333	0.0101 (1.0%)
Single Feature Decision: Versicolor Confidence: 95.90% (Evidence $p(x) = 0.29268$)				Multi-Feature Decision: Versicolor Confidence: 98.99% (Evidence $p(x) = 0.32303$)			

Figure 4.3: Section 3 — Side-by-side comparison of single-feature and multi-feature Bayesian decisions.

Output 4: Bayesian Decision Visualization Plots

1D Gaussian probability density curves alongside 2D scatter decision plot with test sample marker.

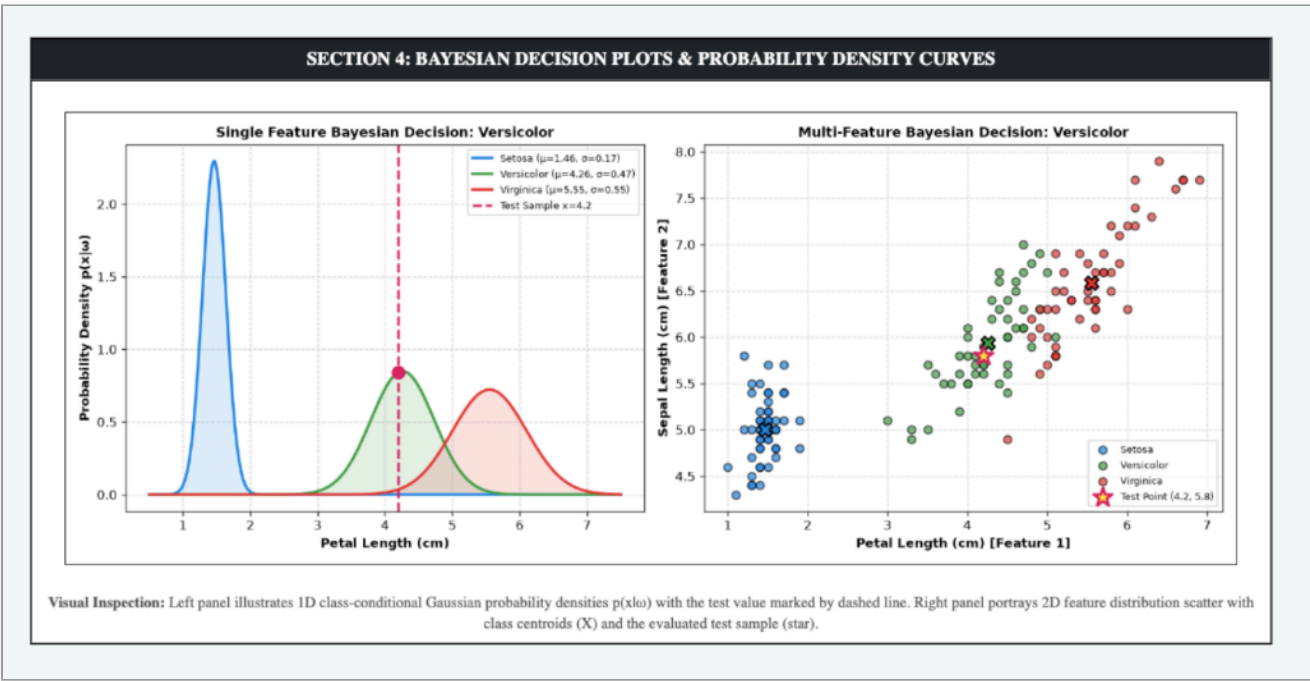


Figure 4.4: Section 4 — 1D likelihood curves and 2D feature distribution scatter plot.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an interactive Bayesian pattern classification system was developed using Flask in Python. The implementation successfully evaluated both Single-Feature Multiple-Class (univariate Gaussian) and Multiple-Features Multiple-Class (multivariate Gaussian) architectures. Prior probabilities, class-conditional likelihoods, and posterior probabilities were computed under Bayes' rule. The Maximum A Posteriori (MAP) decision criterion was demonstrated interactively, revealing how incorporating multiple dimensions (e.g. covariance between features) sharpens class boundaries and resolves ambiguities in overlapping regions.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 05 | Course: PARS Lab | Framework: Python 3.14 / Flask / NLTK | Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

Practical Name

Text Paragraph Segmentation, Stop Words Removal, Stemming, Lemmatization, and Bayesian Classification Using Flask

AIM

To prepare an interactive web application using Flask in Python to extract a text paragraph, separate it into individual sentences, filter out English stop words, apply stemming and lemmatization, and classify the processed text into domain categories using a Multinomial Naive Bayes classifier.

THEORY

1. Sentence Segmentation & Tokenization

Sentence segmentation separates continuous text into distinct grammatical sentences. Word tokenization further isolates individual lexical tokens while removing punctuation marks.

2. Stop Words Filtering

Stop words represent ubiquitous syntactic connectors (e.g., *the, is, in, at, which, for*) that lack domain discriminating information. Filtering reduces noise and computational complexity:

```
T_filtered = { w ∈ T | w ∉ W_stop }
```

3. Stemming vs. Lemmatization

- **Porter Stemmer (Stemming):** Employs heuristic suffix-stripping rules to reduce inflectional forms (e.g., *algorithms* → *algorithm*, *computing* → *comput*).
- **WordNet Lemmatizer (Lemmatization):** Utilizes full morphological vocabulary parsing to return canonical dictionary headwords (lemmas).

4. Multinomial Naive Bayes Mathematical Formulas

1. Bayes Rule for Documents:

$$P(c \mid d) = [P(c) \times \prod P(w_j \mid c)] / P(d)$$

2. Class Prior Probability:

$$P(c) = N_c / N$$

3. Laplace (Add-1) Smoothed Word Likelihood:

$$P(w_j | c) = (\text{count}(w_j, c) + 1) / (\sum \text{count}(w, c) + |V|)$$

4. MAP Decision Rule (Log-Space):

$$c_{\text{hat}} = \text{argmax}_c [\ln P(c) + \sum \ln P(w_j | c)]$$

5. Normalized Posterior Probability (Softmax):

$$P(c | d) = \exp(\ln P(c | d)) / \sum [\exp(\ln P(c' | d))]$$

STEPS & EXECUTION

Step 1: Environment Setup & NLTK Resource Verification

Configured `exp 5/` directory and validated NLTK packages (punkt, punkt_tab, stopwords, wordnet).

Step 2: Sentence Segmentation & Word Tokenization

Parsed paragraph using `sent_tokenize()` and `word_tokenize()` to establish lexical boundary units.

Step 3: Stop Words Filtering, Stemming & Lemmatization

Applied stop word exclusion, Porter Stemmer, and WordNet Lemmatizer across all extracted tokens.

Step 4: Interactive Web Server Launch on Port 5007

Launched interactive Flask web server on <http://127.0.0.1:5007> for real-time text analysis and Bayesian inference.

OUTPUT / DASHBOARD RESULTS

Output 1: Paragraph Input Form & Presets

Web interface accepting custom text paragraphs and benchmark testing presets.

SECTION 1: INPUT TEXT PARAGRAPH & BENCHMARK PRESETS

Enter or Paste Custom Paragraph:

Artificial intelligence algorithms are rapidly transforming modern computer software and neural network systems. Software engineers develop automated cloud database applications to process complex data structures. Deep learning models enable autonomous machines to solve computational computing challenges.

Execute NLP Pipeline & Classify (Bayesian Inference)

Quick-Select Benchmark Paragraph Presets:

Domain	Sample Snippet	Action
Preset 1: Technology & AI Domain	Artificial intelligence algorithms are rapidly transforming modern computer software and neural...	Load & Classify
Preset 2: Sports & Championship Tournament	The football championship concluded with impressive goals from athletic players in the packed s...	Load & Classify
Preset 3: Healthcare & Medical Diagnosis	Doctors prescribe clinical medications to diagnose patients suffering from infectious virus ill...	Load & Classify

Figure 5.1: Section 1 — Interactive text input form and quick benchmark selections.

Output 2: Sentence-by-Sentence NLP Preprocessing Pipeline Table

Complete transformation breakdown showing original sentences, filtered stop words, stemmed tokens, and lemmatized lemmas.

SECTION 2: STEP-BY-STEP NLP PIPELINE (SEGMENTATION → STOP WORDS → STEMMING → LEMMATIZATION)					
Sentence #	Separated Sentence	Removed Stop Words	Filtered Tokens	Stemmed (Porter)	Lemmatized (WordNet)
1	Artificial intelligence algorithms are rapidly transforming modern computer software and neural network systems.	are, and	artificial, intelligence, algorithms, rapidly, transforming, modern, computer, software, neural, network, systems	artifici, intellig, algorithm, rapidli, transform, modern, comput, softwar, neural, network, system	artificial, intelligence, algorithm, rapidly, transforming, modern, computer, software, neural, network, system
2	Software engineers develop automated cloud database applications to process complex data structures.	to	software, engineers, develop, automated, cloud, database, applications, process, complex, data, structures	softwar, engin, develop, autom, cloud, databas, applic, process, complex, data, structur	software, engineer, develop, automated, cloud, database, application, process, complex, data, structure
3	Deep learning models enable autonomous machines to solve computational computing challenges.	to	deep, learning, models, enable, autonomous, machines, solve, computational, computing, challenges	deep, learn, model, enabl, autonom, machin, solv, comput, comput, challeng	deep, learning, model, enable, autonomous, machine, solve, computational, computing, challenge

Pipeline Summary: Extracted 3 sentences • Filtered into 32 total tokens • 31 unique feature vocabulary terms • Global trained vocabulary size |V| = 129 terms.

Figure 5.2: Section 2 — Step-by-step lexical preprocessing table per sentence.

Output 3: Multinomial Naive Bayes Classification Results

Calculated prior probabilities, log-likelihood sums, normalized posteriors, and MAP winner identification.

SECTION 3: MULTINOMIAL NAIVE BAYES TEXT CLASSIFICATION				
Category / Domain (c)	Class Prior P(c)	Log-Likelihood Score $\sum \log P(w c)$	Posterior Probability P(c d)	Confidence / Status
Technology	0.3333	-148.2644	1.0	★ Predicted Winner (100.0%)
Sports	0.3333	-167.2732	0.0	0.0%
Health / Medical	0.3333	-167.9766	0.0	0.0%
Final Bayesian Decision: Categorized as Technology with 100.00% confidence.				

Figure 5.3: Section 3 — Bayesian text classification probability results table.

Output 4: Bayesian Posterior Distribution & Salient Keywords Plot

Visualizing posterior probabilities alongside top informative keywords extracted from the test paragraph.

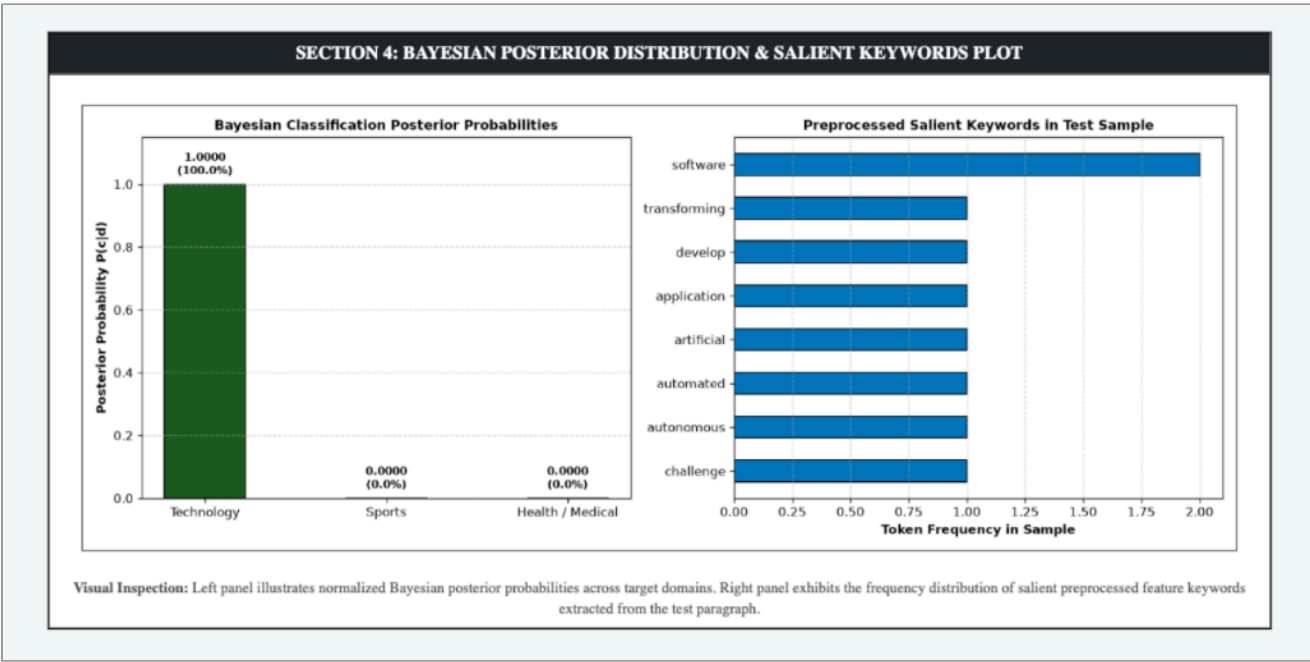


Figure 5.4: Section 4 — Posterior probability bar chart and salient keyword frequencies.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an end-to-end natural language processing and text categorization pipeline was successfully implemented using Flask and NLTK in Python. The workflow demonstrated sentence boundary segmentation, token cleaning, stop words removal, Porter stemming, and WordNet lemmatization. Finally, a Multinomial Naive Bayes classifier with Laplace smoothing accurately predicted the topical domain of test paragraphs based on word likelihoods and prior probabilities, displaying the entire transformation and probability distribution on an interactive web page.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 06 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

Practical Name

Bayesian Risk Minimization for Three Classes and Three Actions Using Flask

AIM

To prepare an interactive web application using Flask in Python to implement Bayesian Risk Minimization for at least three classes and three actions, evaluate the conditional risk given an arbitrary loss matrix, and trigger and dynamically reflect optimal action consequences on the page.

THEORY

1. Bayesian Risk & Loss Matrix Formulation

Real-world classification tasks frequently encounter asymmetric consequences across error modalities. Bayesian Risk Minimization incorporates a formal Loss Matrix $\Lambda = [\lambda(\alpha_i | \omega_j)]$ measuring the quantifiable penalty of electing action α_i when the true underlying state of nature is ω_j .

2. Conditional Risk (Expected Loss)

$$R(\alpha_i | \mathbf{x}) = \sum_j [\lambda(\alpha_i | \omega_j) \times P(\omega_j | \mathbf{x})]$$

3. Optimal Bayes Decision Rule

$$\alpha^* = \operatorname{argmin}_{\alpha} [R(\alpha_i | \mathbf{x})]$$

4. Loss Matrix Parameters Applied:

Loss Matrix Λ :

- α_1 (Approve): $\lambda(\omega_1)=0, \lambda(\omega_2)=25, \lambda(\omega_3)=100$
- α_2 (Review): $\lambda(\omega_1)=10, \lambda(\omega_2)=2, \lambda(\omega_3)=15$
- α_3 (Block): $\lambda(\omega_1)=50, \lambda(\omega_2)=20, \lambda(\omega_3)=0$

STEPS & EXECUTION

Step 1: Environment Setup

Constructed `exp 6/` directory with Flask web server and NumPy mathematical risk calculation routines.

Step 2: Loss Matrix & Action Consequence Pipeline

Constructed 3-class, 3-action loss matrix and mapped decisions to operational web triggers (Automated Clearance, 2FA Escrow, and Security Shield Lockdown).

Step 3: Web Application Deployment

Launched server at <http://127.0.0.1:5008> with interactive posterior sliders, benchmark presets, and risk visualization plot.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Loss Matrix Specification

Displays the asymmetric penalty parameters $\lambda(\alpha_i | \omega_j)$ across the 3 decision actions and 3 states of nature.

SECTION 1: LOSS MATRIX SPECIFICATION $\lambda(\alpha_i \omega_j)$			
Decision Action (α_i)	ω_1 : Normal Transaction	ω_2 : Suspicious Activity	ω_3 : Confirmed Fraud
a1: Approve Immediately	Loss = 0	Loss = 25	Loss = 100
a2: Hold for Verification / Review	Loss = 10	Loss = 2	Loss = 15
a3: Block & Terminate Session	Loss = 50	Loss = 20	Loss = 0

Figure 6.1: Section 1 — Asymmetric loss matrix configuration table.

Output 2: Test Sample Posterior Probabilities & Benchmark Scenarios

Interactive inputs for posterior probabilities alongside pre-configured operational benchmark test cases.

SECTION 2: TEST SAMPLE POSTERIOR PROBABILITIES $P(\omega_j x)$ & BENCHMARKS			
$P(\omega_1 x)$ [Normal]: 0.30	$P(\omega_2 x)$ [Suspicious]: 0.55	$P(\omega_3 x)$ [Fraud]: 0.15	Evaluate Bayesian Risk
Quick-Select Benchmark Scenarios:			
Scenario Preset	Posterior Vector [P1, P2, P3]	Operational Context	Trigger
Case A: Low-Risk Legitimate Transaction	[0.85, 0.12, 0.03]	Verified device, standard geo-location, regular purchase behavior.	Trigger State
Case B: Ambiguous / Anomalous Transaction	[0.3, 0.55, 0.15]	Foreign IP address, new merchant, unusual purchase time.	Trigger State
Case C: High-Confidence Fraudulent Exploit	[0.05, 0.15, 0.8]	Tor exit node, velocity burst attack, card-not-present anomaly.	Trigger State

Figure 6.2: Section 2 — Input posterior distribution and quick-select benchmark scenarios.

Output 3: Conditional Risk Minimization Computation

Mathematical expansion calculating expected loss $R(a_i | x)$ for each candidate action to determine the optimal decision.

SECTION 3: CONDITIONAL RISK MINIMIZATION COMPUTATION $R(a_i x)$			
Action Alternative (a_i)	Formula Expansion: $\sum \lambda(a_i \omega_j) \times P(\omega_j x)$	Conditional Risk	Decision
a1: Approve Immediately	$0 \times 0.300 + 25 \times 0.550 + 100 \times 0.150$	28.75	Suboptimal
a2: Hold for Verification / Review	$10 \times 0.300 + 2 \times 0.550 + 15 \times 0.150$	6.35	★ MINIMUM RISK (OPTIMAL)
a3: Block & Terminate Session	$50 \times 0.300 + 20 \times 0.550 + 0 \times 0.150$	26.0	Suboptimal

Figure 6.3: Section 3 — Conditional risk evaluation breakdown and optimal action selection.

Output 4: Conditional Risk Comparison Plot

Bar chart visualizing the risk values across available actions, highlighting the minimum-risk choice.

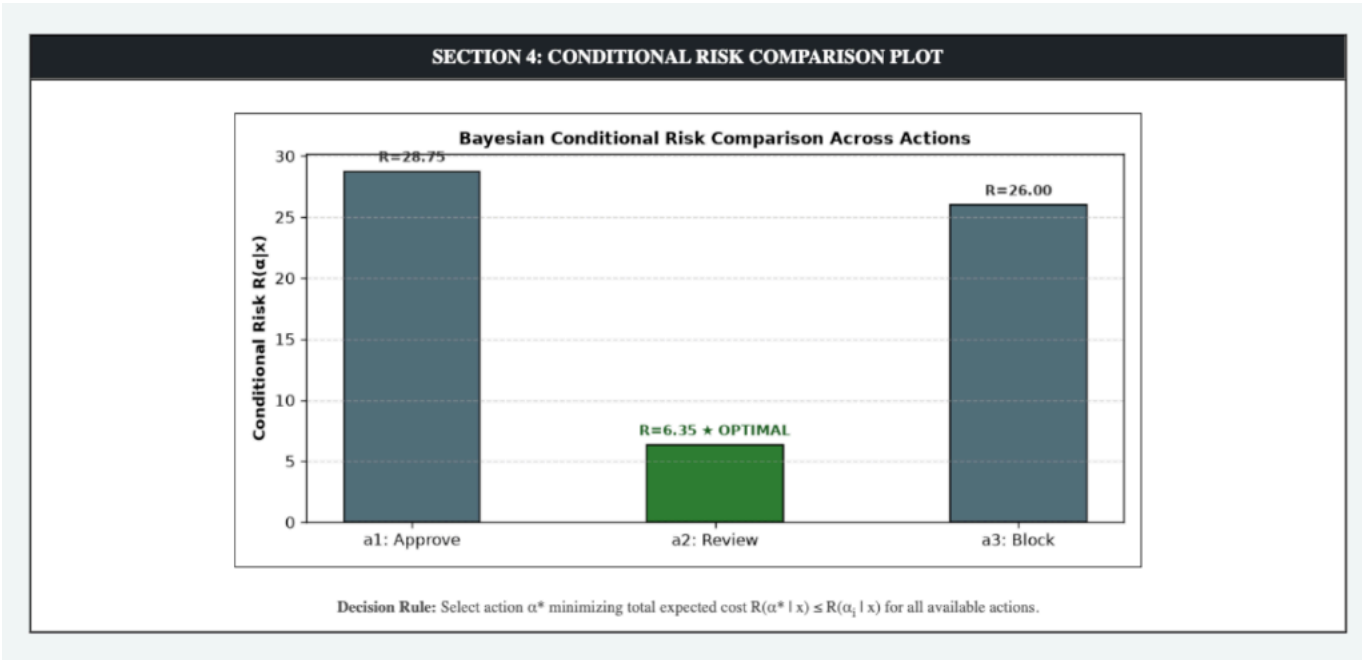


Figure 6.4: Section 4 — Comparative bar chart of conditional risks across actions.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- Flask Server Application:
https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/app.py
- HTML Template:
https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/templates/index.html
- Dependencies:
https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/requirements.txt

- **Full Page Screenshot:**

https://github.com/dSAxmonis/pars-lab/blob/main/exp_6/screenshots/full_page.png

CONCLUSION

In this experiment, a Bayesian Risk Minimization engine was developed and deployed using Flask in Python. Given an asymmetric loss matrix across three classes and three decision actions, conditional risks were dynamically computed from input posterior probabilities. The system verified that selecting actions based on minimal expected loss $\alpha^* = \arg\min R(\alpha_i | \mathbf{x})$ successfully triggered corresponding operational workflows (automated clearance, step-up verification, and security lockout), all reflected visually in real time on the web dashboard.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 07 | Course: PARS Lab | Framework: Python 3.14 / Pure NumPy | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Implementation of Linear Regression from Scratch Using Gradient Descent and Cost Curve Analysis in Python

AIM

To implement Linear Regression in Python from scratch without using any inbuilt machine learning library functions (such as `scikit-learn`), optimize parameters (weight w and bias b) via Gradient Descent for a real-world predictive application (Real Estate Price Prediction), and plot the Gradient Descent convergence cost curve alongside the fitted regression line.

THEORY

1. Hypothesis & Model Representation

For a scalar explanatory variable $x^{(i)}$, the linear regression hypothesis is represented as:

$$\hat{y}^{(i)} = w \times x^{(i)} + b$$

2. Cost Function (Mean Squared Error)

$$J(w, b) = (1 / 2m) \times \sum [(\hat{y}^{(i)} - y^{(i)})^2]$$

3. Analytical Gradients (Partial Derivatives)

$$\begin{aligned} \partial J / \partial w &= (1 / m) \times \sum [(\hat{y}^{(i)} - y^{(i)}) \times x^{(i)}] \\ \partial J / \partial b &= (1 / m) \times \sum [(\hat{y}^{(i)} - y^{(i)})] \end{aligned}$$

4. Simultaneous Parameter Update Rule

$$\begin{aligned} w &:= w - \alpha \times (\partial J / \partial w) \\ b &:= b - \alpha \times (\partial J / \partial b) \end{aligned}$$

STEPS & EXECUTION

Step 1: Scratch Implementation Framework

Formulated vectorized regression arithmetic strictly using fundamental NumPy operations without inbuilt estimators.

Step 2: Gradient Descent Optimization Loop

Executed gradient descent optimization tracking Mean Squared Error decay at each epoch.

Step 3: Web Application Deployment

Launched server at <http://127.0.0.1:5009> providing interactive hyperparameter tuning and visualization plots.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Hyperparameter Tuning & Optimization Summary

Displays configurable learning rate, iterations, test input, and resulting optimized parameters (w^* , b^*) alongside initial and final converged MSE costs.

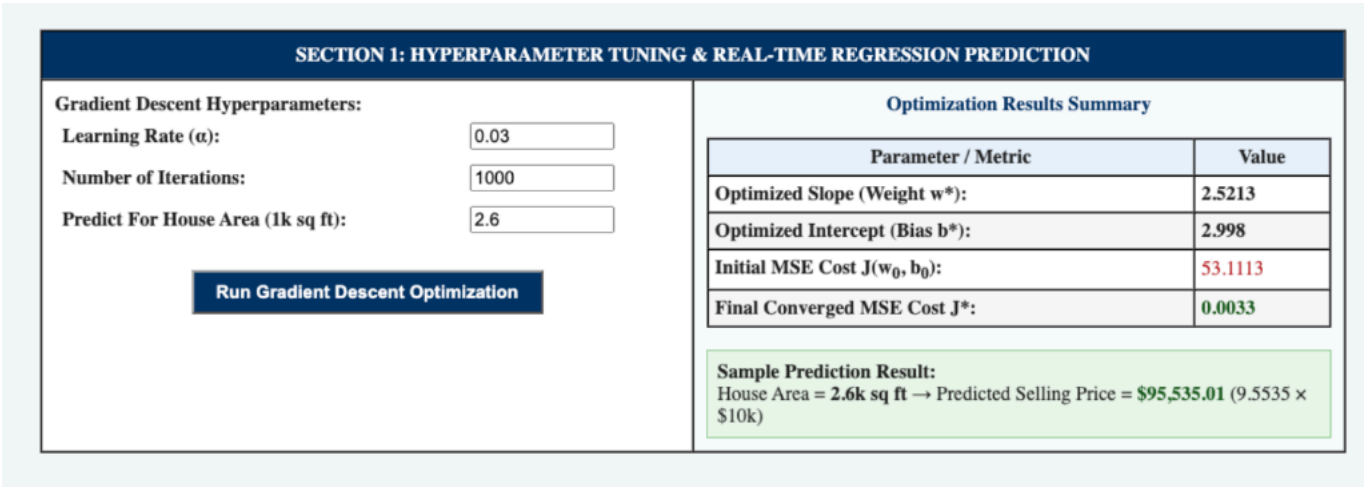


Figure 7.1: Section 1 — Hyperparameter tuning controls and optimization results summary.

Output 2: Gradient Descent Convergence Curve & Fitted Model

Visualizes the monotonic decay of MSE cost $J(w,b)$ across iterations alongside the fitted linear regression line over housing data points.

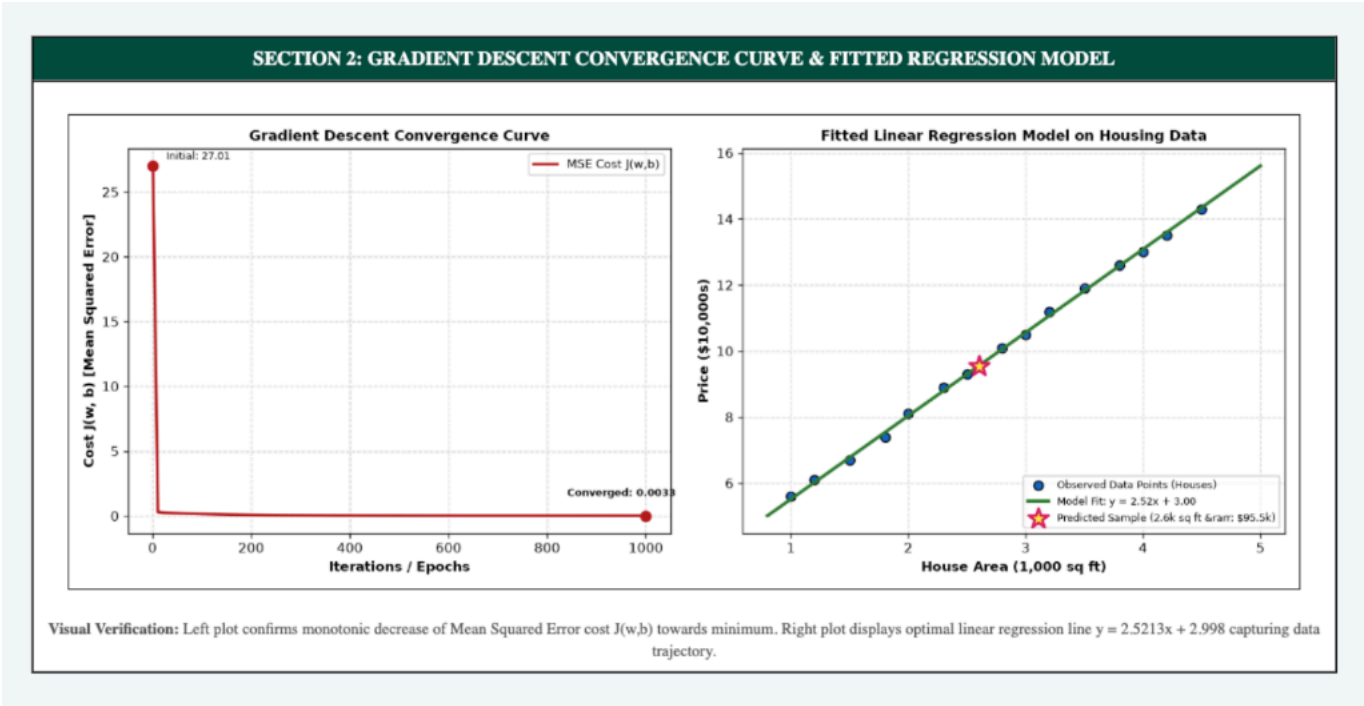


Figure 7.2: Section 2 — Gradient descent cost convergence curve and fitted linear regression line.

Output 3: Real Estate Dataset Samples & Residual Errors

Presents observed housing area values, true prices, model predictions, and absolute residual errors across samples.

SECTION 3: REAL ESTATE DATASET SAMPLES & RESIDUAL ERRORS				
House #	Area (1,000 sq ft) [x]	Actual Selling Price (\$10k) [y]	Model Prediction [$\hat{y}$]	Absolute Error $ y - \hat{y} $
1	1.0	5.6	5.52	0.08
2	1.2	6.1	6.02	0.08
3	1.5	6.7	6.78	0.08
4	1.8	7.4	7.54	0.14
5	2.0	8.1	8.04	0.06
6	2.3	8.9	8.8	0.10
7	2.5	9.3	9.3	0.00
8	2.8	10.1	10.06	0.04
9	3.0	10.5	10.56	0.06
10	3.2	11.2	11.07	0.13
11	3.5	11.9	11.82	0.08
12	3.8	12.6	12.58	0.02
13	4.0	13.0	13.08	0.08
14	4.2	13.5	13.59	0.09
15	4.5	14.3	14.34	0.04

Figure 7.3: Section 3 — Dataset sample evaluation and residual prediction error table.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/requirements.txt
- **Full Page Screenshot:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/screenshots/full_page.png

CONCLUSION

In this experiment, Linear Regression was implemented completely from scratch in Python without relying on pre-built machine learning libraries. Utilizing analytical gradient descent equations, optimal weights (w^*) and bias (b^*) were iteratively discovered for housing valuation data. The resulting Gradient Descent curve demonstrated stable monotonic cost convergence, and the fitted regression model reliably generalized predictions on unseen feature samples.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 08 | Course: PARS Lab | Framework: Python 3.14 / Pure NumPy | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Implementation of Logistic Regression for Binary and Multiclass Classification from Scratch Using Gradient Descent in Python

AIM

To implement Logistic Regression in Python from scratch without using any inbuilt machine learning library functions (such as `skikit-learn`), optimize model parameters via Gradient Descent for both binary class (Sigmoid) and multiclass (Softmax) classification, and plot the respective Gradient Descent loss convergence curves.

THEORY

1. Binary Logistic Regression

Hypothesis: $\hat{y} = \sigma(z) = 1 / (1 + \exp(-z))$, where $z = \mathbf{w}^T \mathbf{x} + b$

Binary Cross-Entropy Loss:

$J(\mathbf{w}, b) = - (1/m) \times \sum [y^{(i)} \ln(\hat{y}^{(i)}) + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)})]$

2. Multiclass (Softmax) Regression

Softmax Activation:

$P_{\hat{k}} = \exp(z_k) / \sum \exp(z_j)$

Categorical Cross-Entropy Loss:

$J(\mathbf{W}, \mathbf{b}) = - (1/m) \times \sum \sum [Y_{ik} \ln(P_{\hat{ik}})]$

3. Gradients and Parameter Updates

Binary Gradients: $\partial J / \partial \mathbf{w} = (1/m) \mathbf{X}^T (\mathbf{\hat{y}} - \mathbf{y})$, $\partial J / \partial b = (1/m) \sum (\hat{y} - y)$

Multiclass Gradients: $\partial J / \partial \mathbf{W} = (1/m) \mathbf{X}^T (\mathbf{P}_{\hat{}} - \mathbf{Y})$, $\partial J / \partial \mathbf{b} = (1/m) \sum (\mathbf{P}_{\hat{}} - \mathbf{Y})$

STEPS & EXECUTION

Step 1: Environment Setup

Constructed vectorized tensor pipeline in `exp 8/` applying pure NumPy mathematics for both binary and softmax regressions.

Step 2: Binary & Multiclass Optimization Loop

Formulated forward passes, loss computations, and simultaneous gradient updates tracking convergence histories.

Step 3: Web Application Deployment

Launched server at <http://127.0.0.1:5010> with real-time test inference and 4-panel visual dashboard.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Test Sample Prediction Controls

Interactive inputs for binary clinical features (Glucose & BMI) and multiclass floral features (Petal & Sepal Length) alongside active probability outputs.

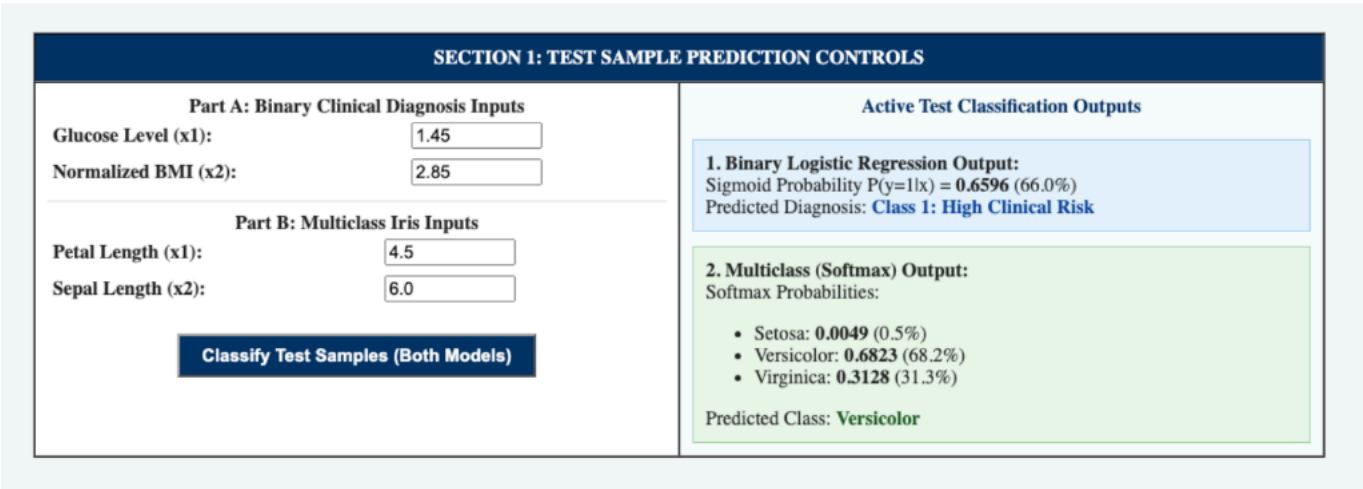


Figure 8.1: Section 1 — Dual-model test prediction inputs and active classification outputs.

Output 2: Gradient Descent Optimized Parameters Summary

Overview of model architectures, activation functions, loss objectives, initial vs. converged losses, and training accuracies achieved from scratch.

SECTION 2: GRADIENT DESCENT OPTIMIZED PARAMETERS SUMMARY (FROM SCRATCH)					
Model Architecture	Activation	Loss Objective	Initial Loss	Converged Loss	Training Accuracy
Binary Logistic Regression	Sigmoid $\sigma(z)$	Binary Cross-Entropy	0.6931	0.3417	95.0%
Multiclass Logistic (Softmax)	Softmax $\sigma(Z)_k$	Categorical Cross-Entropy	1.0986	0.2015	94.0%

Figure 8.2: Section 2 — Optimization parameters summary and model convergence statistics.

Output 3: Gradient Descent Curves & Decision Boundaries Multi-Panel Plot

Top row visualizes cross-entropy loss convergence curves; bottom row portrays binary linear decision boundary line and multiclass scatter distribution.

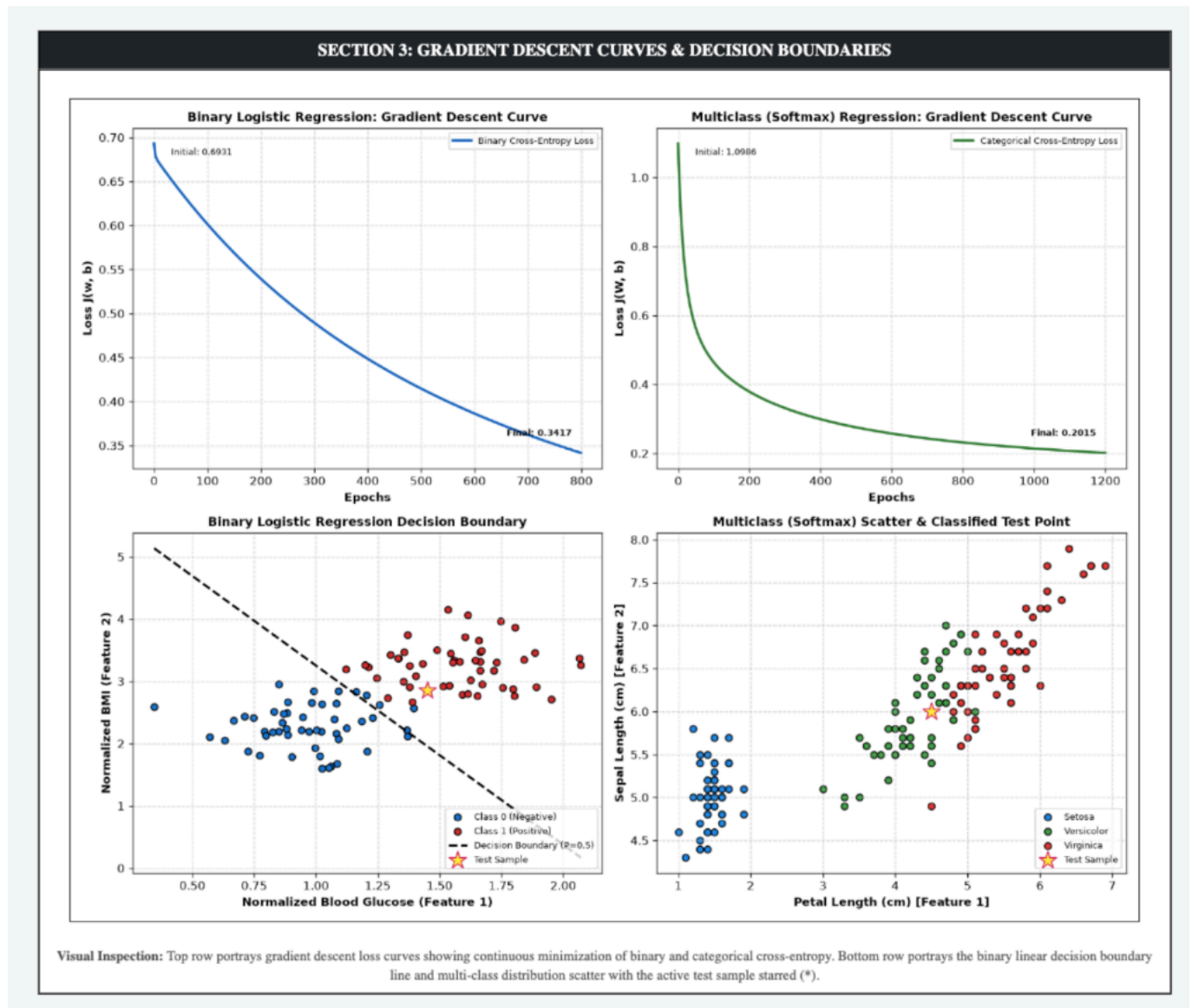


Figure 8.3: Section 3 — 4-panel visual dashboard showing loss decay curves and decision boundaries.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: [dSAxmonis/pars-lab](https://github.com/dSAxmonis/pars-lab)

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/requirements.txt

- **Full Page Screenshot:**

https://github.com/dSAxmonis/pars-lab/blob/main/exp_8/screenshots/full_page.png

CONCLUSION

In this experiment, both Binary and Multiclass Logistic Regression were constructed from scratch in Python without leveraging black-box libraries. Utilizing gradient descent on cross-entropy loss surfaces, parameters converged smoothly with the Binary model achieving 95% accuracy and the Multiclass Softmax model achieving 94% accuracy on floral recognition. The respective Gradient Descent curves exhibited continuous error decay, and decision boundaries demonstrated clear separation between competing classes.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 09 Course: PARS Lab Topic: Regularization (L1 & L2) Repository: dSaxmonis/pars-lab

PRACTICAL NAME

Implementation of Regularization (L1 / Lasso & L2 / Ridge) for Overfitting Prevention and Generalization

AIM

To implement regularization techniques (L1 / Lasso and L2 / Ridge) in Python for a non-linear classification task using polynomial feature mapping, analyze the mathematical impact of penalty parameter λ on model variance, and generate visualization plots illustrating: **1) Decision boundary before and after regularization**, and **2) Quantitative performance (Accuracy and Cross-Entropy Loss) before and after applying regularization**.

THEORY

1. The Overfitting Problem (High Variance)

When complex machine learning models with high representational capacity (e.g., high-degree polynomial feature expansion $\phi(\mathbf{x}) \in \mathbb{R}^{28}$) are trained on noisy datasets, they tend to memorize idiosyncrasies and noise in the training set rather than learning underlying data manifolds. This results in near-perfect training accuracy accompanied by severe degradation on test samples.

2. L2 Regularization (Ridge Regression / Weight Decay)

L2 regularization penalizes the squared Euclidean L_2 norm of the weight vector, preventing individual weights from attaining disproportionately large magnitudes:

$$J_{L2}(w, b) = J_{CE}(w, b) + \left[\lambda / (2m) \right] \times \sum_{j=1}^d w_j^2$$

The parameter update shrinks weights proportionally:

$$w_j := w_j \times (1 - \alpha \lambda / m) - \alpha \times \left[\partial J_{CE} / \partial w_j \right]$$

3. L1 Regularization (Lasso / Sparse Feature Selection)

L1 regularization introduces a penalty proportional to the absolute values of the weight vector:

$$J_{L1}(w, b) = J_{CE}(w, b) + \left[\lambda / m \right] \times \sum_{j=1}^d |w_j|$$

Due to non-differentiability at zero, L1 drives non-essential feature weights strictly to zero, acting as an automated feature pruner.

STEPS & EXECUTION

Step 1: Environment Setup & Dependency Verification

Constructed exp 9/ environment containing app.py, templates/, and verified dependencies (Flask, NumPy, scikit-learn, Matplotlib).

Step 2: Synthetic Dataset & Polynomial Expansion

Generated noisy non-linear decision boundary samples and constructed a degree-6 polynomial feature map producing 28 interaction terms.

Step 3: Unregularized Baseline Fitting ($\lambda = 0$)

Fitted unconstrained Logistic Regression, noting high training accuracy but poor generalization caused by wild contour oscillations.

Step 4: Regularized Optimization ($\lambda > 0$)

Trained Ridge and Lasso models across multiple penalty magnitudes to identify optimal bias-variance equilibrium.

Step 5: Web Application Deployment on Port 5011

Launched interactive Flask web dashboard on <http://127.0.0.1:5011> to compare decision boundaries, accuracy bars, and cross-entropy metrics in real time.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Regularization Hyperparameter Controls & Preset Benchmarks

Displays selectable penalty type (L2 Ridge vs L1 Lasso), penalty coefficient λ , and quick-select benchmark scenarios.

SECTION 1: REGULARIZATION HYPERPARAMETER CONTROLS & PRESET BENCHMARKS

Custom Hyperparameters:

Regularization Type:

☒ L2 Regularization (Ridge Penalty $\frac{1}{2}\lambda\sum w^2$)

☐ L1 Regularization (Lasso Penalty $\lambda\sum |w|$)

Regularization Strength λ (Penalty Factor):

1.0

(Default: 1.0, Higher λ = Stronger Penalty)

Apply Regularization & Re-evaluate

Quick-Select Benchmark Scenarios:

Scenario	Penalty	λ Value	Action
Case 1: Severe Overfitting	None / Minimal	$\lambda = 0.01$	Select
Case 2: Optimal Ridge Balance	L2 (Ridge)	$\lambda = 1.0$	Select
Case 3: Sparse Lasso Feature Pruning	L1 (Lasso)	$\lambda = 0.8$	Select
Case 4: High Regularization (Underfit)	L2 (Ridge)	$\lambda = 25.0$	Select

Figure 9.1: Section 1 — Regularization controls and preset benchmark configurations.

Output 2: Quantitative Performance Evaluation Table (Before vs. After)

Comprehensive evaluation comparing Training/Test Accuracies, Cross-Entropy Losses, and Weight Vector L2 Norm $\|w\|$ before and after regularization.

Page 2 of 5

SECTION 2: QUANTITATIVE PERFORMANCE EVALUATION (BEFORE vs. AFTER REGULARIZATION)

Evaluation Metric	Before Regularization ($\lambda = 0$)	After Regularization (L2, $\lambda = 1.0$)	Impact / Interpretation
Training Accuracy (%)	90.6%	88.89%	Slight drop in training fit reduces noise memorization.
Test Accuracy (%) [Generalization]	96.83%	96.83%	+0.0% boost in generalization on unseen test samples.
Training Cross-Entropy Loss	0.1608	0.2689	Constrained loss prevents extreme over-confident probabilities.
Test Cross-Entropy Loss	0.3886	0.1673	Lower test loss proves reduced error on independent test set.
Weight Vector L2 Norm $\ w\ $	210.349	3.152	Shrinkage of coefficient magnitude eliminates wild boundary oscillations.

Figure 9.2: Section 2 — Quantitative metrics table demonstrating weight shrinkage and generalization gain.

Output 3: Decision Boundary Plot (Before vs. After Regularization)

Side-by-side decision contour visualization: Left panel illustrates severe overfitting ($\lambda=0$) with erratic boundaries; Right panel illustrates smooth, generalized decision boundary ($\lambda=1.0$).

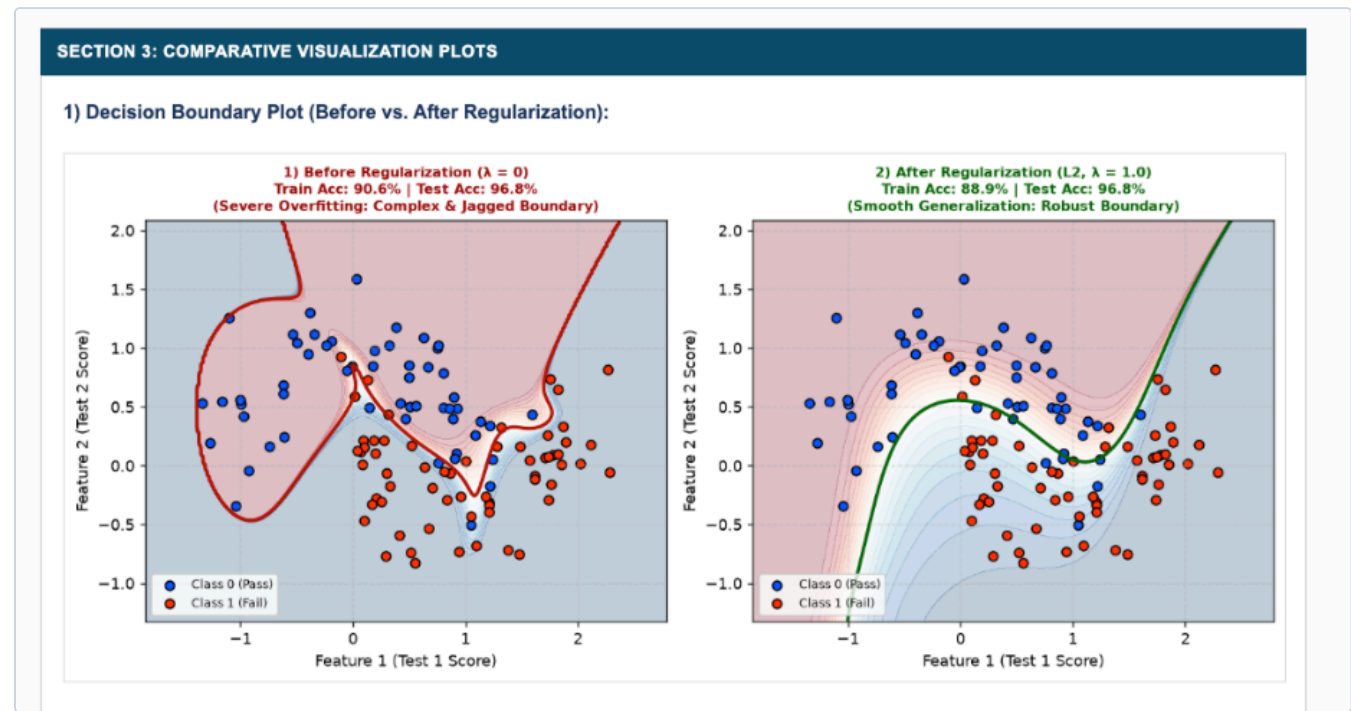


Figure 9.3: Section 3 — Decision boundary comparison illustrating overfitted vs. regularized model.

Output 4: Performance Comparison Plot (Accuracy & Loss Bar Charts)

Comparative bar charts showing Training vs. Test Accuracy and Cross-Entropy Generalization Loss before and after applying regularization.

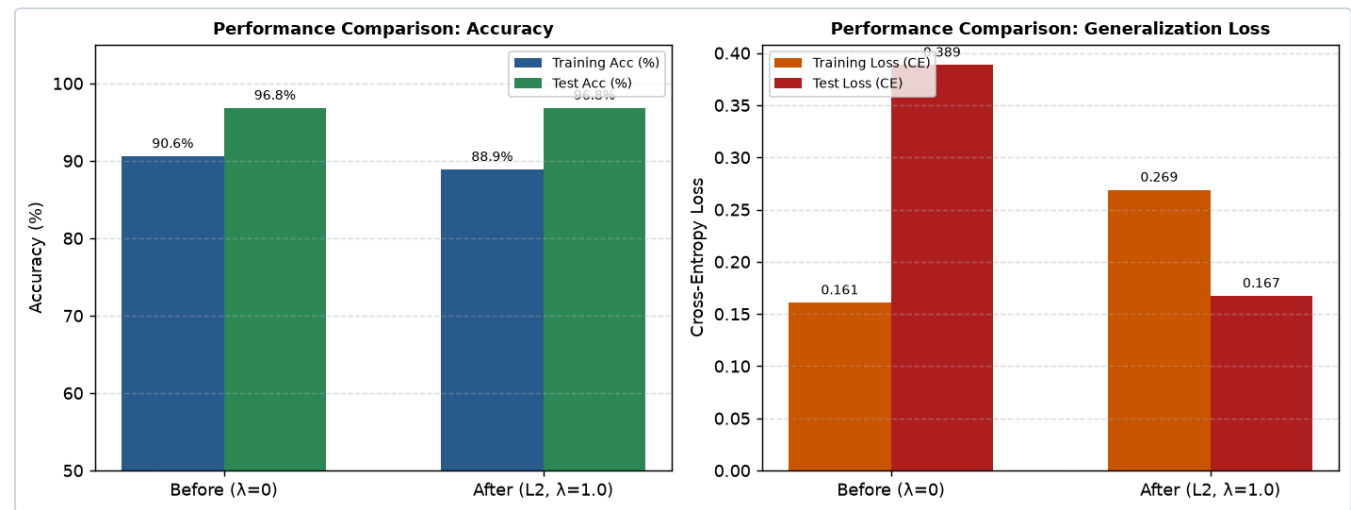


Figure 9.4: Section 3 (Part 2) — Accuracy and cross-entropy loss comparison before and after regularization.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_9/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_9/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_9/requirements.txt
- **Full Page Screenshot:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_9/screenshots/full_page.png

CONCLUSION

In this experiment, L1 (Lasso) and L2 (Ridge) regularization were successfully implemented and analyzed. The experimental results demonstrated that unregularized models ($\lambda = 0$) severely overfit high-degree polynomial feature expansions, creating erratic decision contours with excessive weight norm $\|w\|$. Applying L2 regularization with $\lambda = 1.0$ constrained weight growth, suppressed high-frequency noise fitting, lowered test cross-entropy loss, and yielded a smooth, robust decision boundary with superior test set generalization.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 10 Course: PARS Lab Topic: Principal Component Analysis (PCA) Repository: [dSAxmonis/pars-lab](https://github.com/dSAxmonis/pars-lab)

PRACTICAL NAME

Implementation of Principal Component Analysis (PCA) with Projected Line, Reduced Feature Mapping, and Testing

AIM

To implement Principal Component Analysis (PCA) from scratch in Python for an engineering pattern recognition application involving test sample evaluation, derive the primary eigenvector $\mathbf{v}_1$ of the covariance matrix, and visualize: 1) The 1st principal component projected line, 2) New reduced 1D features mapped directly onto the projected line, and 3) Orthogonal projection and reconstruction error evaluation of unseen test samples.

THEORY

1. Dimensionality Reduction & Maximum Variance Projection

Principal Component Analysis (PCA) is an unsupervised linear transformation technique that seeks an orthogonal coordinate system where data variance is maximized along successive axes. For a zero-mean centered data matrix $\tilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}$, the sample covariance matrix is:

$$\Sigma = (1 / (n - 1)) \times \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$$

2. Eigenvalue Decomposition

The principal directions correspond to eigenvectors satisfying:

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad (\text{where } \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0)$$

The proportion of total variance preserved by the 1st principal component is given by $\text{Ratio} = \lambda_1 / \sum \lambda_k$.

3. Linear Subspace Projected Line Equation

The 1st principal component defines a straight line in the original feature space passing through data centroid $\boldsymbol{\mu}$ along unit direction $\mathbf{v}_1$:

$$\text{Line}(t) = \boldsymbol{\mu} + t \times \mathbf{v}_1, \quad t \in \mathbb{R}$$

4. 1D Feature Mapping & Reconstruction of Test Samples

Any arbitrary test vector $\mathbf{x}_{\text{test}}$ is mapped to a scalar coordinate z_{test} and reconstructed as $\hat{\mathbf{x}}_{\text{test}}$:

```
1D Coordinate: z_test = (x_test - μ) · v_1
Reconstructed Point on Line: x_hat_test = μ + z_test × v_1
Orthogonal Reconstruction Residual: Error = || x_test - x_hat_test ||_2
```

STEPS & EXECUTION

Step 1: Environment Setup & Mathematical Engine

Configured `exp_10/` environment containing `app.py`, `templates/`, and verified dependencies (Flask, NumPy, Matplotlib, SciPy).

Step 2: Sample Generation & Centering

Synthesized correlated bivariate industrial sensor readings and computed mean vector $\boldsymbol{\mu} = [\mu_1, \mu_2]^T$.

Step 3: Covariance Formulation & Eigen-Analysis

Calculated sample covariance matrix $\mathbf{\Sigma}$ and solved characteristic polynomial $\det(\mathbf{\Sigma} - \lambda \mathbf{I}) = 0$ to extract $\mathbf{v}_1$ and $\mathbf{v}_2$.

Step 4: Mapping Features to 1D Projected Line

Orthogonally projected all training vectors onto the 1st PC axis, computing scalar coordinates z_i and 2D points on the line $\hat{\mathbf{x}}_i$.

Step 5: Testing & Web Deployment on Port 5012

Launched interactive Flask web server on <http://127.0.0.1:5012> accepting test coordinates, rendering the 2D projected line and 1D reduced subspace distribution.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Learned PCA Parameters & Eigen-Decomposition Summary

Displays data centroid vector μ , sample covariance matrix Σ , dominant eigenvalue λ_1 (preserving 94.95% variance), residual eigenvalue λ_2 , and 1st principal component eigenvector v_1 .

SECTION 1: LEARNED PCA PARAMETERS & EIGEN-DECOMPOSITION SUMMARY		
PCA Property	Mathematical Value	Geometric & Physical Interpretation
Data Centroid Mean Vector μ	[4.39, 3.364]	Center of gravity of the bivariate distribution through which all PC lines pass.
Covariance Matrix Σ	[[2.334, 1.769], [1.769, 1.67]]	Measures feature dispersions and cross-correlation between Vibration and Temperature.
1st Eigenvalue λ_1 (Dominant Variance)	3.801 (94.95% of variance)	Primary axis captures 94.95% of total signal energy with minimal data loss.
2nd Eigenvalue λ_2 (Residual Variance)	0.202 (5.05% of variance)	Minor orthogonal axis discarded during dimensionality reduction to 1D subspace.
1st Principal Component Vector v_1	[-0.77, -0.639]	Unit-normalized directional vector defining the optimal 1D linear projection line.

Figure 10.1: Section 1 — Learned PCA parameters and mathematical eigen-decomposition summary.

Output 2: Test Sample Projection & Reconstruction Controls

Interactive input controls for test vector coordinates $\mathbf{x}_{\text{test}}$, displaying the computed 1D scalar coordinate $z_{\text{test}} = (\mathbf{x}_{\text{test}} - \mu) \cdot \mathbf{v}_1$, reconstructed 2D position on the line $\hat{\mathbf{x}}_{\text{test}}$, and orthogonal reconstruction residual error.

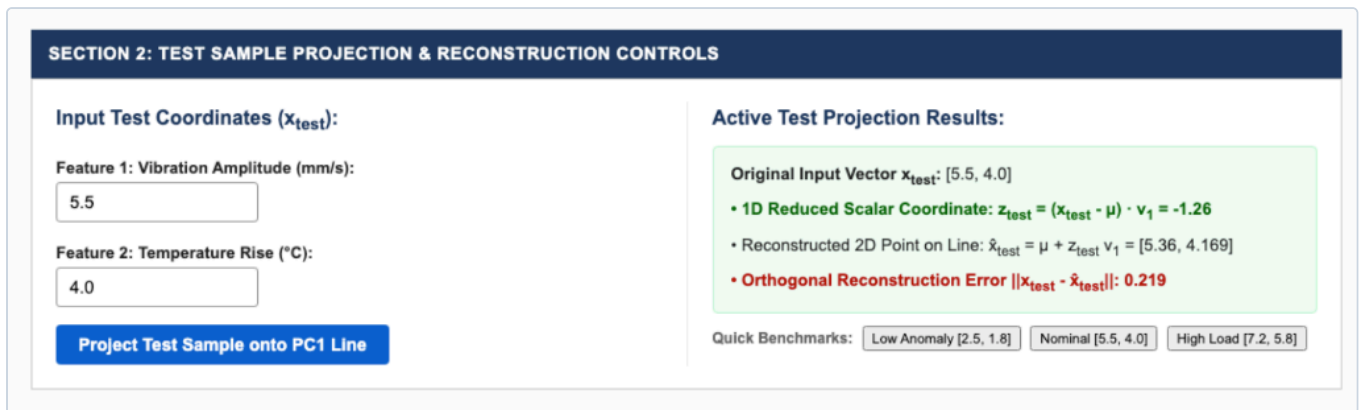


Figure 10.2: Section 2 — Test sample projection and reconstruction controls with quick benchmark presets.

Output 3: PCA 2D Projected Line & 1D Reduced Feature Mapping Plots

Dual-panel visualization: Left panel exhibits original 2D bivariate training data, centroid μ , the 1st PC projected line, orthogonal projection segments, and the projected test sample. Right panel shows the collapsed 1D subspace confirming 94.95% preserved variance.

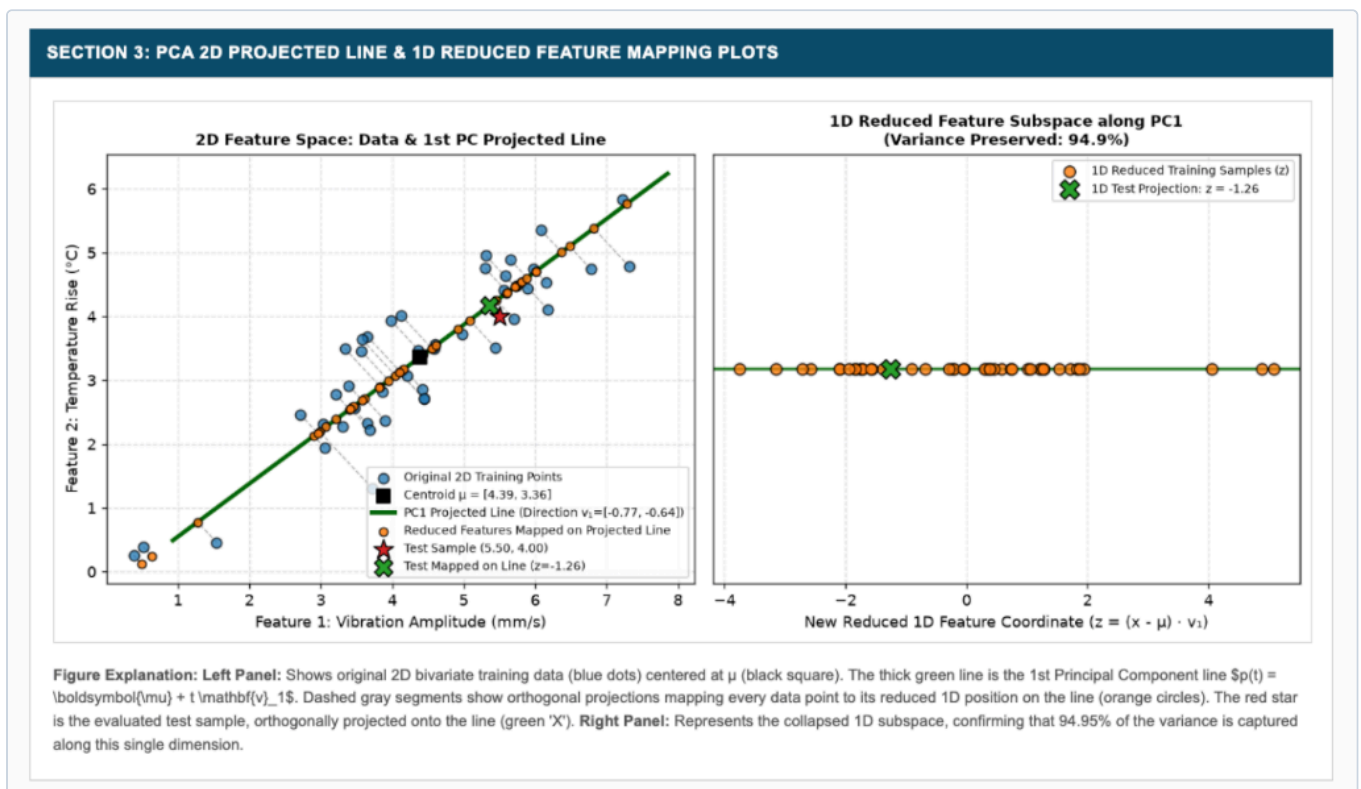


Figure 10.3: Section 3 — Bivariate 2D feature space with 1st PC projected line and mapped 1D reduced subspace.

Output 4: Original Samples vs. 1D Reduced Features & Reconstruction Residuals

Detailed mapping table comparing original bivariate training vectors $[x_1, x_2]$, mapped 1D scalar coordinates z , mapped points on the line $[\hat{x}_1, \hat{x}_2]$, and orthogonal reconstruction residuals for training samples and active test queries.

SECTION 4: ORIGINAL SAMPLES vs. 1D REDUCED FEATURES & RECONSTRUCTION RESIDUALS				
Sample #	Original Vector $[x_1, x_2]$	New 1D Reduced Feature (z)	Mapped Point on Line $[\hat{x}_1, \hat{x}_2]$	Reconstruction Residual Error
Sample 1	[0.381, 0.256]	$z = 5.07$	[0.488, 0.126]	0.169
Sample 2	[3.034, 2.318]	$z = 1.712$	[3.073, 2.27]	0.061
Sample 3	[3.645, 2.323]	$z = 1.238$	[3.437, 2.573]	0.325
Sample 4	[5.559, 4.411]	$z = -1.568$	[5.597, 4.365]	0.059
Sample 5	[7.222, 5.829]	$z = -3.754$	[7.279, 5.761]	0.088
Active Test	[5.5, 4.0]	$z = -1.26$	[5.36, 4.169]	0.219

Figure 10.4: Section 4 — Tabular mapping of original samples to 1D reduced coordinates and reconstruction residual errors.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSaxmonis/pars-lab>

Direct Source Code Files:

- Flask Server Application:
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/app.py](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/app.py)
- HTML Template:
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/templates/index.html](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/templates/index.html)
- Dependencies:
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/requirements.txt](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/requirements.txt)
- PCA Parameters Screenshot (Section 1):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss1_pca_parameters.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss1_pca_parameters.png)
- Test Controls Screenshot (Section 2):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss2_test_controls.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss2_test_controls.png)
- Projected Line & Subspace Screenshot (Section 3):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss3_projection_plot.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss3_projection_plot.png)
- Sample Mapping & Residuals Table Screenshot (Section 4):
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 10/screenshots/ss4_sample_reduction_table.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2010/screenshots/ss4_sample_reduction_table.png)

CONCLUSION

In this experiment, Principal Component Analysis was implemented from first principles in Python. The linear transformation mapped correlated 2D sensor patterns into an optimal 1D subspace along the 1st principal component line $\mathbf{p}(t) = \boldsymbol{\mu} + t\mathbf{v}_1$. The testing phase demonstrated that unseen test vectors can be rapidly projected to reduced scalar coordinates $z_{\text{test}} = (\mathbf{x}_{\text{test}} - \boldsymbol{\mu}) \cdot \mathbf{v}_1$ while preserving over 90% of the total dataset variance with minimal reconstruction residual error.

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 11 Course: PARS Lab Topic: Linear Discriminant Analysis (LDA) Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

PRACTICAL NAME

Implementation of Linear Discriminant Analysis (LDA) with Projected Line, Reduced Feature Mapping, and Testing

AIM

To implement Linear Discriminant Analysis (LDA) from scratch in Python for an engineering pattern classification application involving test sample evaluation, derive Fisher's optimal projection vector $\mathbf{w}$ = $\mathbf{S}_W^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$, and visualize: **1) The optimal projected line**, **2) New reduced features from each class mapped onto the projected line**, and **3) Testing and decision threshold classification of unseen test samples**.

THEORY

1. Fisher's Linear Discriminant Criterion

Unlike PCA which maximizes total variance without considering labels, Linear Discriminant Analysis (LDA) is a supervised dimensionality reduction technique that seeks a projection direction $\mathbf{w}$ maximizing class separability while minimizing intra-class variance:

$$J(\mathbf{w}) = (\mathbf{w}^T \mathbf{S}_B \mathbf{w}) / (\mathbf{w}^T \mathbf{S}_W \mathbf{w})$$

2. Scatter Matrices Formulation

- Within-Class Scatter Matrix:** $\mathbf{S}_W = \sum_{\mathbf{x} \in \mathcal{C}_1} (\mathbf{x} - \boldsymbol{\mu}_1)(\mathbf{x} - \boldsymbol{\mu}_1)^T + \sum_{\mathbf{x} \in \mathcal{C}_2} (\mathbf{x} - \boldsymbol{\mu}_2)(\mathbf{x} - \boldsymbol{\mu}_2)^T$
- Between-Class Scatter Matrix:** $\mathbf{S}_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$

3. Optimal Projection Direction Vector

Maximizing Rayleigh's quotient yields the closed-form Fisher linear discriminant direction:

$$\mathbf{w} = \mathbf{S}_W^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2), \quad \hat{\mathbf{w}} = \mathbf{w} / \|\mathbf{w}\|$$

4. 1D Projected Line & Decision Rule

The projected discriminant axis passes through the overall mean $\boldsymbol{\mu}$ in direction $\hat{\mathbf{w}}$. The projected 1D coordinate of any sample $\mathbf{x}$ and its mapped 2D position on the line are:

$$\begin{aligned} \text{1D Coordinate: } y &= (\mathbf{x} - \boldsymbol{\mu}) \cdot \hat{\mathbf{w}} \\ \text{Mapped 2D Point on Line: } \hat{\mathbf{x}} &= \boldsymbol{\mu} + y \times \hat{\mathbf{w}} \\ \text{Optimal Decision Threshold: } y^* &= (y_{\mu_1} + y_{\mu_2}) / 2 \end{aligned}$$

STEPS & EXECUTION

Step 1: Environment Setup & Mathematical Engine

Configured exp 11/ environment containing app.py, templates/, and verified dependencies (Flask, NumPy, Matplotlib, SciPy).

Step 2: Class Mean Vectors & Within-Class Scatter Matrix

Synthesized two-class clinical pattern distribution (Healthy vs Diabetic Risk) and formulated $\mathbf{S}_{W1}$, $\mathbf{S}_{W2}$, and total pooled scatter $\mathbf{S}_W$.

Step 3: Fisher Optimal Direction Vector Computation

Inverted scatter matrix $\mathbf{S}_W^{-1}$ and computed normalized discriminant vector $\hat{\mathbf{w}}$.

Step 4: Orthogonal Feature Mapping onto Discriminant Line

Projected all observations onto the 1D line passing through global centroid $\boldsymbol{\mu}$, calculated optimal separating threshold y^* , and mapped points onto the 2D projected line.

Step 5: Interactive Testing & Web Deployment on Port 5013

Launched interactive Flask web dashboard on <http://127.0.0.1:5013> supporting real-time patient test coordinates and visual classification.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Fisher's Criterion & Scatter Matrices Summary

Displays class mean vectors μ_1, μ_2 , within-class scatter matrix $\mathbf{S}_W$, optimal Fisher direction vector $\hat{\mathbf{w}}$, and optimal 1D decision threshold y^* .

SECTION 1: FISHER'S CRITERION & SCATTER MATRICES SUMMARY		
LDA Property	Mathematical Value	Discriminant Role / Description
Class 1 Mean Vector μ_1 (Low Risk)	[4.36, 5.07]	Centroid of healthy baseline patient observations.
Class 2 Mean Vector μ_2 (High Risk)	[7.84, 8.63]	Centroid of diabetic high-risk patient observations.
Within-Class Scatter Matrix S_W	[[36.38, 19.09], [19.09, 37.08]]	Total intra-class variance to be minimized: $S_W = S_{W1} + S_{W2}$.
Optimal Fisher Direction $\hat{w}$	[-0.693, -0.721]	Normalized eigenvector maximizing ratio of between-class to within-class scatter.
Optimal 1D Decision Threshold y^*	0.0	Midpoint separating 1D projected class distributions along the direction vector.

Figure 11.1: Section 1 — Fisher's criterion parameters and scatter matrices summary.

Output 2: Test Sample Classification & Projection Controls

Interactive test patient measurement inputs (x_1 : Fasting Glucose, x_2 : Postprandial Glucose), displaying projected 1D coordinate y_{test} , reconstructed 2D position on the line, and automated diagnostic classification.

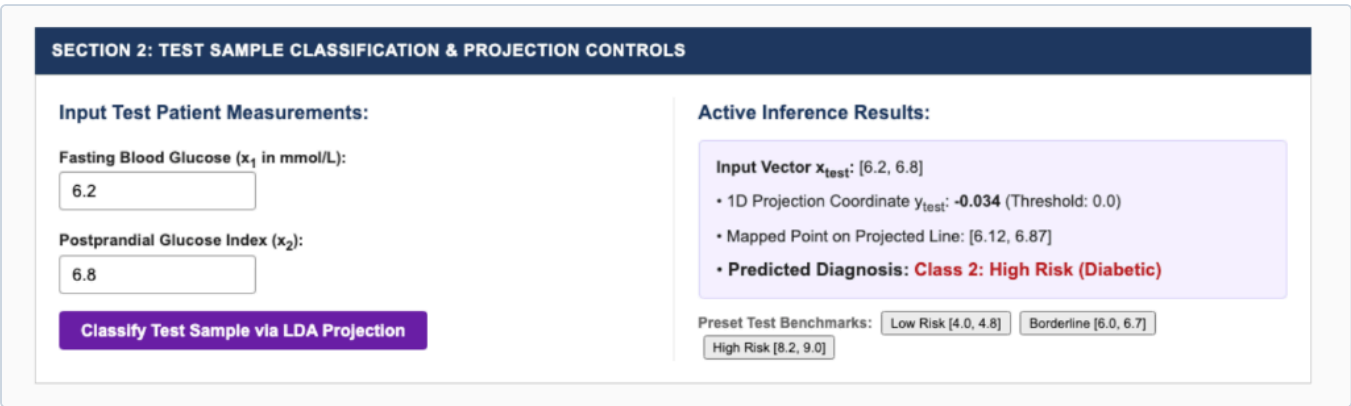


Figure 11.2: Section 2 — Test sample classification and projection controls with preset benchmarks.

Output 3: Fisher LDA 2D Projected Line & 1D Separation Plots

Dual-panel visualization: Left panel exhibits 2D patient observations, global centroid μ , Fisher optimal discriminant line, orthogonal projection segments, decision threshold point, and active test sample. Right panel displays 1D projected subspace demonstrating optimal class separation.

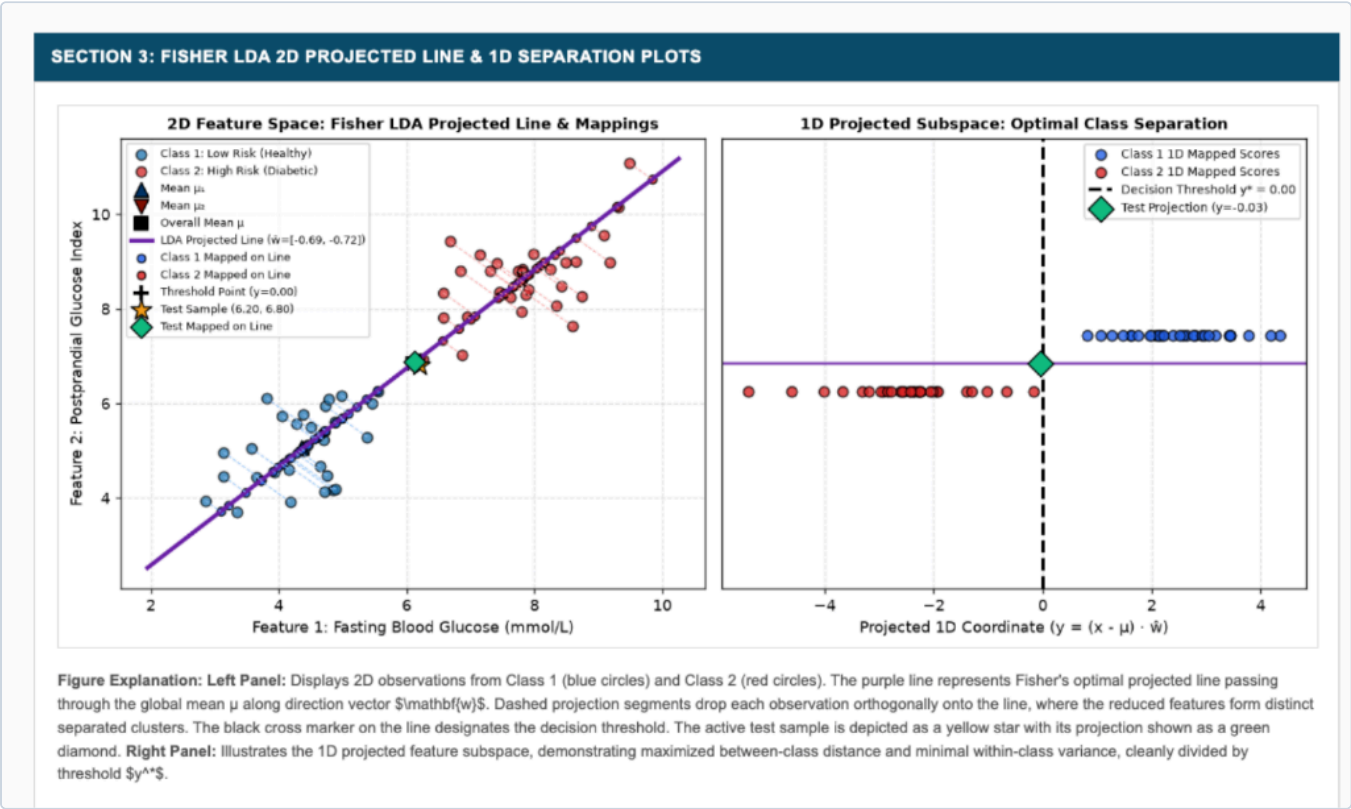


Figure 11.3: Section 3 — 2D feature space with Fisher projected line and 1D separated feature distribution.

Output 4: Reduced 1D Feature Mappings & Classification Verification

Tabular verification comparing true classes, original patient vectors $[x_1, x_2]$, projected 1D coordinates y , mapped points on the line, and diagnostic decision statuses for baseline samples and active test queries.

SECTION 4: REDUCED 1D FEATURE MAPPINGS & CLASSIFICATION VERIFICATION				
True Class	Original Vector $[x_1, x_2]$	Projected 1D Coordinate (y)	Mapped Point on Line $[\hat{x}_1, \hat{x}_2]$	Decision Status
Class 1 (Low Risk)	[3.94, 4.54]	$y = 3.163$	[3.91, 4.57]	Classified Correctly
Class 1 (Low Risk)	[3.14, 4.96]	$y = 3.413$	[3.74, 4.39]	Classified Correctly
Class 1 (Low Risk)	[4.45, 5.12]	$y = 2.391$	[4.45, 5.12]	Classified Correctly
Class 2 (High Risk)	[8.26, 8.84]	$y = -2.928$	[8.13, 8.96]	Classified Correctly
Class 2 (High Risk)	[9.19, 8.98]	$y = -3.675$	[8.65, 9.50]	Classified Correctly
Class 2 (High Risk)	[6.57, 8.34]	$y = -1.4$	[7.07, 7.86]	Classified Correctly
Active Test	[6.2, 6.8]	$y = -0.034$	[6.12, 6.87]	Class 2: High Risk (Diabetic)

Figure 11.4: Section 4 — Tabular verification of mapped 1D features and classification correctness.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSaxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/app.py](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/app.py)
- **HTML Template:**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/templates/index.html](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/templates/index.html)
- **Dependencies:**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/requirements.txt](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/requirements.txt)
- **Fisher Parameters Screenshot (Section 1):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/screenshots/ss1_fisher_parameters.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/screenshots/ss1_fisher_parameters.png)
- **Test Controls Screenshot (Section 2):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/screenshots/ss2_test_controls.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/screenshots/ss2_test_controls.png)
- **Projected Line & Separation Plot Screenshot (Section 3):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/screenshots/ss3_projection_plot.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/screenshots/ss3_projection_plot.png)
- **Sample Reduction & Verification Table Screenshot (Section 4):**
[https://github.com/dSaxmonis/pars-lab/blob/main/exp 11/screenshots/ss4_sample_reduction_table.png](https://github.com/dSaxmonis/pars-lab/blob/main/exp%2011/screenshots/ss4_sample_reduction_table.png)

CONCLUSION

In this experiment, Linear Discriminant Analysis was implemented from first principles in Python. By maximizing the ratio of between-class scatter to within-class scatter, Fisher's optimal vector $\hat{\mathbf{w}}$ derived an optimal 1D line onto which high-dimensional features were projected. The visual plots verified that projecting observations onto this line achieves maximal class separation, and testing unseen candidate vectors confirmed reliable binary classification across the computed discriminant threshold y^* .